

RAPPORT DE STAGE D'INITIATION

Réalisé par : Abir Majdi

Tuteur : Mr. ELAKRAA Soufiane



Elève Ingénieur en 1ère année en Génie de développement numérique et cybersécurité

Stage réalisé au sein de : ***Centre Hospitalier Universitaire Hassan II de Fès (CHU)***

Sujet de stage :

Systeme de détection et de
journalisation des accès anormaux aux
bases de données médicales

Période de stage : du 01/07/2025 à 01/08/2025



REMERCIEMENTS

Je tiens à exprimer ma sincère gratitude envers l'ensemble du corps enseignant du département d'informatique pour leur engagement, leur disponibilité et leur soutien constant tout au long de mon parcours. Leur accompagnement et leurs conseils avisés ont grandement contribué à la réussite de ce stage.

Je souhaite exprimer ma profonde gratitude à mon encadrant, Monsieur **Imadeddine Mountasser**, pour son accompagnement attentif, sa coopération précieuse, ainsi que pour les conseils avisés et les informations qu'il m'a généreusement fournies.

Je tiens également à exprimer ma profonde reconnaissance au **Centre Hospitalier Universitaire (CHU) Hassan II de Fès** pour m'avoir accueilli dans un environnement propice à l'apprentissage et au développement professionnel.

Je remercie particulièrement mon encadrant pédagogique chez **CHU**, Monsieur **Soufiane ELAKRAA**, pour son suivi rigoureux, ses orientations pertinentes et son soutien tout au long de cette expérience.

Leur encadrement et leur confiance ont joué un rôle essentiel dans la réussite de ce stage, et je leur en suis profondément reconnaissant.



RÉSUMÉ

Ce document présente les résultats de mon stage d'initiation d'une durée d'un mois, qui s'est déroulé au sein du Centre Hospitalier Universitaire (CHU) Hassan II de Fès, encadré par de Monsieur Soufiane ELAKRAA.

L'objectif principal de ce stage était le développement d'un système de détection et de journalisation des accès anormaux aux bases de données médicales, dans le cadre de la cybersécurité et de protection des données sensibles. Ce projet cherchait à améliorer la traçabilité et la sécurité des accès aux informations médicales, toute en respectant les bonnes méthodes de gouvernance des données.

À cet effet, j'ai implémenté un code de langage Python avec des bibliothèques telles que le module pandas, mysql.connector, logging et smtplib pour envoyer des e-mails; et un SGBD MySQL sur lequel ma base de données était conçue et implémentée. J'ai développé plusieurs composants, dont :

- Des tables médicales (patients, médecins, consultations, hospitalisations, utilisateurs, logs...),
- Des triggers et vues pour la surveillance des accès,
- Un script d'analyse automatique pour détecter les comportements suspects (accès hors horaires, requêtes anormales, tentatives échouées),
- Un système d'envoi d'alertes par e-mail,
- Et une interface web Flask pour la visualisation et l'analyse des logs, l'export des rapports en format CSV avec envoi d'e-mail en cas d'anomalies détectées et lancement d'un scan sqlmap sur la base

Ce rapport décrit l'environnement hospitalier dans lequel s'est déroulé le stage, les besoins en matière de sécurité des bases de données, ainsi que toutes les étapes techniques du projet, de l'analyse à la mise en œuvre.

Ce stage professionnel dans un domaine critique m'a permis non seulement d'acquérir des connaissances techniques, mais aussi d'avoir un aperçu de la cybersécurité en milieu médical et des réels défis rencontrés lorsqu'on travaille sous pression.



TABLE DES ACRONYMES

Acronyme	Signification	Domaine
CHU	Centre Hospitalier Universitaire	Santé
RSI	Responsable du Système d'Information	Informatique
SI	Système d'Information	Informatique
SGBD	Système de Gestion de Base de Données	Informatique
DME	Dossiers Médicaux Électroniques	Santé/Informatique
RGPD	Règlement Général sur la Protection des Données	Juridique
SQL	Structured Query Language	Informatique
SQLi	SQL Injection	Cybersécurité
CSV	Comma-Separated Values	Informatique
HTML	HyperText Markup Language	Web
HTTP	HyperText Transfer Protocol	Web
HTTPS	HyperText Transfer Protocol Secure	Web
SMTP	Simple Mail Transfer Protocol	Réseau
DNS	Domain Name System	Réseau
URL	Uniform Resource Locator	Web
MCD	Modèle Conceptuel de Données	Base de données
ID	Identifiant	Informatique
CIN	Carte d'Identité Nationale	Administration



Acronyme	Signification	Domaine
MIME	Multipurpose Internet Mail Extensions	Email
TLS	Transport Layer Security	Sécurité
GET/POST	Méthodes HTTP	Web



Remerciements	2
Résumé	3
Table des acronymes	4
Introduction	7
1. Contexte général du projet	8
1.1 Profil du Centre Hospitalier Universitaire Hassan II de Fès	8
1.2 Présentation du projet	11
2. Étude théorique	13
2.1 Introduction à la cybersécurité médicale	13
2.2 Concepts de journalisation et détection d'anomalies	13
2.3 Types d'accès anormaux en milieu hospitalier	14
2.4 Technologies utilisées	14
3. Travaux réalisés	19
3.1 Mise en place de la base de données sécurisée	19
3.2 Développement de l'outil d'analyse Python	24
3.3 Déploiement de l'application web Flask	37
3.4 Tests et scénarios d'attaques simulées	51
4. Résultats obtenus et évaluation	59
1. Pertinence du système dans un environnement hospitalier	59
2. Limites et axes d'amélioration	59
3. Apports du stage	60
Conclusion générale	61



INTRODUCTION

Du 01/07/2025 au 01/08/2025, j'ai fait un stage d'une durée d'un mois dans le CHU Hassan II de Fès, au Maroc. Ce stage s'est déroulé dans le service informatique ayant comme mission la gestion et la protection des SI hospitaliers.

Avec la forte croissance des systèmes numériques et l'importance accrue des données médicales sensibles, la sécurité des systèmes d'information est devenue une priorité majeure. En effet, les attaques visant les bases de données médicales peuvent compromettre la confidentialité, l'intégrité et la disponibilité des informations importantes sur les patients et les professionnels de santé. Cela place les hôpitaux à de sérieux risques en ce qui concerne la cybercriminalité.

Dans ce contexte, mes tâches de stage ont inclus la création et l'exécution d'un système de détection et de journalisation des accès anormaux aux bases de données médicales. L'objectif principal était de surveiller les requêtes effectuées sur la base de données, de détecter les comportements suspects (comme les accès hors horaires, non autorisés ou en grand nombre), d'écrire les événements dans des journaux bien organisés, et d'alerter tout de suite les administrateurs si une anomalie arrive.

Ce projet m'a permis d'approfondir mes connaissances en sécurité informatique, en développement Python, en gestion de bases de données MySQL, ainsi qu'en analyse de logs. J'ai été parfaitement guidé par Monsieur Soufiane ELAKRAA, ainsi que par toute l'équipe informatique du CHU, que je remercie pour leur aide.

Ce rapport est structuré en quatre parties :

- Une première partie consacrée à la présentation du CHU Hassan II de Fès, au contexte général du projet et à la problématique de sécurité abordée.
- Une deuxième partie présentant les outils et technologies de détection d'anomalies étudiés, ainsi que les solutions envisagées.
- Une troisième partie détaillant la réalisation technique du projet, de la conception à l'implémentation.
- Une quatrième partie portant sur les résultats obtenus, les difficultés rencontrées et les perspectives d'amélioration.

1. CONTEXTE GENERAL DU PROJET

1.1 PROFIL DU CENTRE HOSPITALIER UNIVERSITAIRE HASSAN II DE FES

Le Centre Hospitalier Universitaire (CHU) Hassan II de Fès est un établissement de santé de renom situé à Fès, au Maroc, à proximité de la faculté de médecine et de pharmacie. Il s'étend sur une superficie de 12 hectares et relève du ministère de la Santé [4].



Figure 1.1– CHU Hassan II de Fès

1.1.1 HISTORIQUE DU CHU HASSAN II

Le Centre Hospitalier Universitaire (CHU) Hassan II de Fès a été officiellement inauguré par Sa Majesté le Roi Mohammed VI le 14 janvier 2009. Sa création répond à la volonté de renforcer l'offre de soins spécialisés dans la région Fès-Meknès et d'accompagner la formation médicale et paramédicale. Conçu comme une structure moderne répondant aux standards internationaux, le CHU Hassan II assure à la fois des missions de soins, d'enseignement et de recherche. Depuis son ouverture, il s'est imposé comme un pôle régional de référence, doté d'infrastructures avancées et jouant un rôle essentiel dans la prise en charge des patients, la formation des futurs médecins et le développement de la recherche scientifique.

1.1.2 STRUCTURES ET SERVICES HOSPITALIERS

Le CHU est organisé autour d'une direction centrale et de plusieurs établissements hospitaliers spécialisés, tels que :

- ✓ L'Hôpital des Spécialités
- ✓ L'Hôpital Mère-Enfant
- ✓ L'Hôpital d'Oncologie
- ✓ L'Hôpital Omar Drissi
- ✓ L'Hôpital Ibn Al Hassan

La direction du CHU se compose de différentes divisions de gestion, à savoir :

- ✓ Secrétariat Général
- ✓ Division des Affaires Médicales et des Soins Infirmiers
- ✓ Division des Ressources Humaines, de la Formation et de la Coopération
- ✓ Division des Affaires Financières, Logistiques et de la Maintenance
- ✓ Service Informatique et Statistiques

1.1.3 SERVICE INFORMATIQUE

Le service informatique du CHU est structuré en trois cellules principales :

- ❖ Cellule de développement et du système d'information : responsable de la gestion et de la résolution des problèmes relatifs au système d'information hospitalier.
- ❖ Cellule réseau : chargée de la maintenance et de la supervision du réseau informatique de l'établissement.
- ❖ Cellule télécom : en charge de la gestion et de la maintenance du réseau de téléphonie.

Les missions clés du service informatique incluent :

- Fournir un support technique efficace pour toute anomalie liée au système d'information.
- Assurer le bon fonctionnement du réseau téléphonique interne.
- Réaliser la maintenance des équipements informatiques.
- Superviser le réseau informatique de l'établissement.

1.1.4 ORGANIGRAMME DE LA DIRECTION DU CHU DE FES

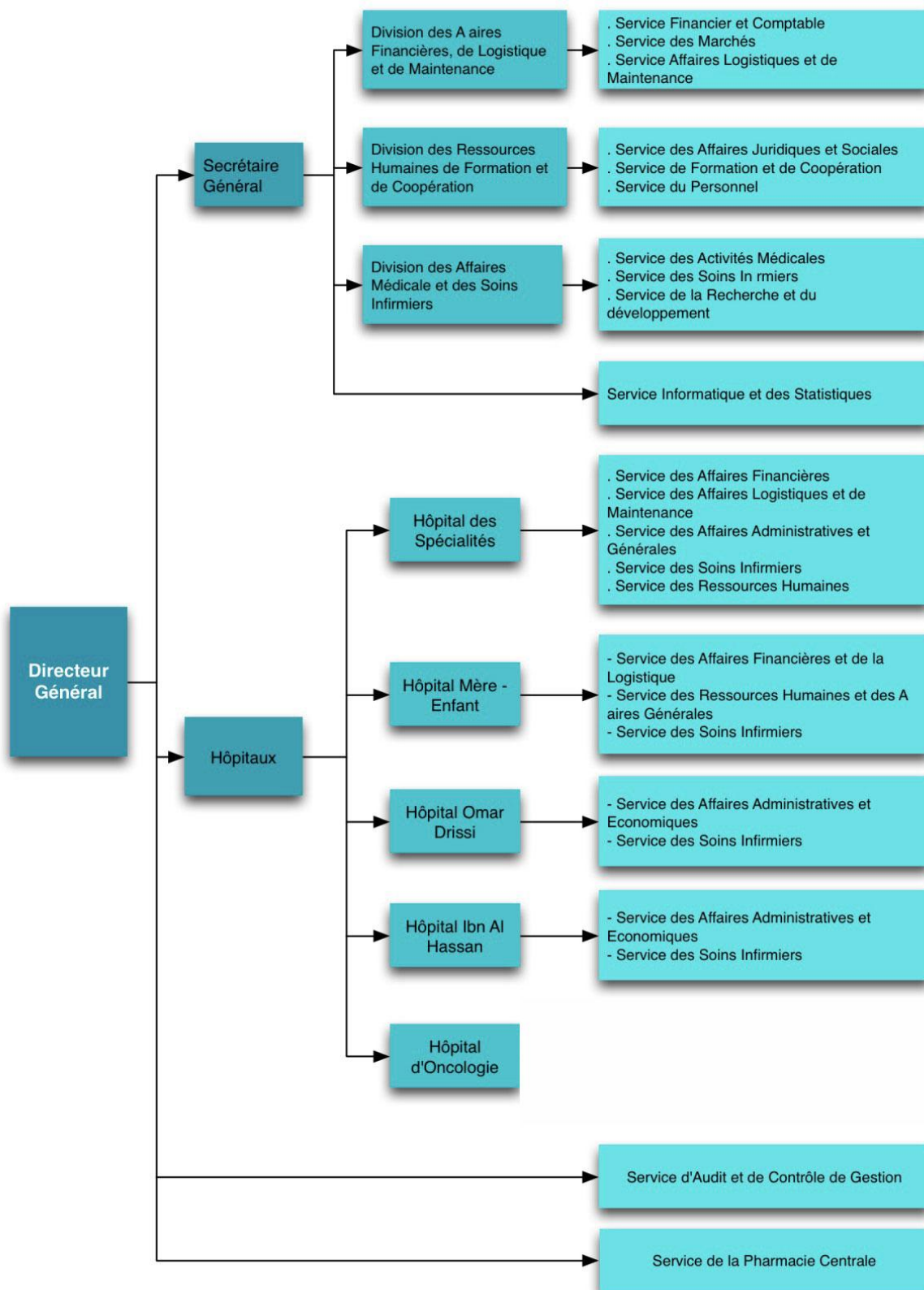


Figure 1.2 : Organigramme de la direction du CHU Hassan II de Fès



1.2 PRESENTATION DU PROJET

1.2.1 PROBLEMATIQUE

De nos jours, les établissements hospitaliers font face à des menaces numériques de plus en plus nombreuses qui visent à pirater les systèmes d'information de ces derniers. Des cas d'accès non autorisés aux données des patients, des consultations hors horaires des dossiers médicaux ou des formes abusives d'accès aux services par les membres du personnel peuvent entraîner des conséquences graves en termes d'obligation juridique et éthique. Cependant, des mécanismes d'analyse automatique sont nécessaires car de nombreux hôpitaux et centres hospitaliers n'ont aucun moyen automatisé pour détecter de tels comportements.

La problématique principale de ce projet repose donc sur la question suivante : **Comment concevoir un système capable d'identifier automatiquement des comportements anormaux dans l'accès aux données médicales, tout en garantissant une traçabilité complète et une réactivité face aux incidents ?**

1.2.2 OBJECTIFS DU PROJET

Le projet vise plusieurs objectifs précis et complémentaires :

- **Surveillance des accès** : Mettre en place un système de journalisation automatique de toutes les requêtes effectuées sur la base de données.
- **Détection des anomalies** : Définir des règles de détection d'accès anormaux (accès hors horaires, accès non autorisé à certains types de données, pics inhabituels d'activité, etc.).
- **Traçabilité** : Conserver une trace fiable des actions effectuées par chaque utilisateur via des logs horodatés.
- **Alerte en temps réel** : Développer un mécanisme d'alerte automatique (par email, par exemple) lorsqu'un comportement suspect est détecté.
- **Facilité d'analyse** : Proposer une interface ou un outil permettant de visualiser les anomalies, les tendances et les statistiques d'accès.

1.2.3 METHODOLOGIE ADOPTÉE

Pour répondre à la problématique posée et atteindre les objectifs fixés, une méthodologie agile et structurée a été adoptée :



- **Analyse des besoins** : Étude du fonctionnement du système d'information hospitalier et identification des points critiques en matière d'accès aux données.
- **Conception de la base de données** : Ajout de nouvelles tables (logs d'accès, anomalies détectées, seuils de détection, sessions utilisateurs, etc.) et création de vues pour faciliter les analyses.
- **Développement de l'outil de surveillance** : Utilisation de Python avec les bibliothèques mysql.connector, pandas, logging et smtplib pour le traitement des données et la détection d'anomalies.
- **Mise en place de triggers SQL** : Automatisation de certaines vérifications et journalisations directement au niveau de la base de données.
- **Tests et validation** : Injection de scénarios types (accès suspects simulés) afin de vérifier l'efficacité du système de détection.
- **Documentation et amélioration continue** : Rédaction d'une documentation technique et collecte des retours des utilisateurs pour ajuster les seuils ou enrichir les règles de détection.



2. ÉTUDE THEORIQUE

2.1 INTRODUCTION A LA CYBERSECURITE MEDICALE

La cybersécurité médicale, ou e-santé, est un exemple parfait d'une discipline née de vingt et unième siècle. En effet, le besoin crucial de protéger les systèmes d'information hospitaliers est apparu avec la multiplication des menaces numériques de la dernière décennie. De nos jours, les hôpitaux modernes fonctionnent presque entièrement grâce aux technologies de l'information. Les médecins utilisent des DME (dossiers médicaux électroniques) pour écrire leurs réceptions, des systèmes de planification pour organiser le workflow des soins, des serveurs pour stocker les résultats d'analyses et communiquer avec un autre service, où les malades attendent des procédures plus intensives. Comme on peut le voir, toute cette infrastructure digitale nécessite une protection contre les ransomwares, les fuites de données, et les attaques des accès non autorisés. Ce dernier est le plus dangereux, car les données contenues dans les hôpitaux sont parmi les plus sensibles. Au début, je mentionnais fugacement les droits des patients, et le droit à la vie privée est sans doute plus vaste. Protéger les données des cybercriminelles est donc monumentalement crucial. Cela se fait par la défense complète des trois piliers de sécurité des informations : la confidentialité, l'intégrité et la disponibilité. Pour y arriver, il faut assurer une surveillance continue, une détection des comportements suspects et une réaction rapide face aux incidents [9].

2.2 CONCEPTS DE JOURNALISATION ET DETECTION D'ANOMALIES

La **journalisation**, ou logging, est une action qui consiste à enregistrer les activités sur un système informatique comme les systèmes SIEM [3]. En l'occurrence, un tel système au sein de l'hôpital est le suivant : accès aux bases de données médicales, voir des dossiers des patients, ajout ou suppression de données, connexion et déconnexion d'utilisateurs, etc. Ces journaux (logs) sont nécessaires pour assurer la traçabilité des activités, notamment pour détecter les activités suspectes et fournir des preuves en cas d'incident [8].

La **détection d'anomalies**, d'autre part, implique l'analyse de ces logs pour identifier les comportements qui ne relèvent pas de la façon dont le système devrait fonctionner, tels que l'accès aux dossiers du patient en dehors des heures de travail, la consultation des dossiers en l'absence de la justification médicale, et activité en temps normal. Ces anomalies peuvent être à l'origine des tentatives d'intrusion, des erreurs humaines ou des abus internes. Les méthodes en utilisant des statistiques, des règles métier et parfois des approches basées sur le "Machine Learning" peuvent être utilisées pour détecter ces écarts. Le couplage de la journalisation et de la détection d'anomalies forme une couche essentielle de la cybersécurité proactive dans les hôpitaux.



2.3 LES TYPES D'ACCES ANORMAUX EN MILIEU HOSPITALIER

Dans un contexte hospitalier, il existe plusieurs définitions qui prévalent pour les accès anormaux aux bases de données médicales. Premièrement, les **accès non autorisés** qui ne correspondent pas à l'autorisation d'un individu pour visionner les informations consultées se produisent. Par exemple, au lieu d'une connexion aux données administratives, il y a une infrastructure personnelle accédant aux données cliniques. Deuxièmement, **les accès en dehors de leur temps de travail**. Il serait étrange que l'utilisateur soit connecté à 2 heures du matin lorsque ses plans de travail ne le justifient pas. Dire cela pourrait signifier un vol de session ou une tentative d'accès non autorisé.

Dans le même ordre de comportement, il est possible d'inclure l'**accès de masse** ou **répété** à de longs d'aucun dossier. C'est souvent un comportement presque inhumain, et c'est un signe assez évident qu'il s'agit d'une collecte illégale ou de toute activité automatisée. Troisièmement, et il existe certainement bien plus de combinaisons possibles de tels comportements, il est possible d'inclure les comportements internes visant **l'abus des privilèges**. C'est quand les professionnels accèdent à des dossiers à des fins personnelles ou par simple curiosité, plutôt qu'en lien avec leur mission médicale. Ce faisant, il est possible de garantir l'éthique, la confidentialité des soins et la conformité aux réglementations telles que le RGPD (Règlement Général sur la Protection des Données).

2.4 TECHNOLOGIES UTILISEES

Le projet repose sur un ensemble de technologies complémentaires qui permettent de développer un système fiable de **détection et de journalisation des accès anormaux** aux bases de données médicales. Le choix de chaque technologie a été guidé par la nécessité d'assurer l'interopérabilité, la performance et la simplicité de maintenance du système dans un environnement hospitalier.

2.4.1 MySQL

MySQL est le système de gestion de base de données relationnelle utilisé pour stocker l'ensemble des informations médicales, les utilisateurs, les logs d'accès ainsi que les anomalies détectées. Son adoption repose sur sa robustesse, sa compatibilité avec les environnements hospitaliers, et sa capacité à gérer des volumes importants de données structurées.



Dans le cadre du projet, plusieurs tables médicales (patients, médecins, consultations, hospitalisations, infirmiers, etc.) ont été mises en place, en plus de tables spécialisées pour les logs d'accès, les anomalies détectées et les logs requêtes [6]. Des vues, triggers, index et contraintes d'intégrité ont également été définis pour faciliter l'audit, la détection et l'analyse rapide des accès anormaux.

2.4.2 PYTHON ET BIBLIOTHEQUES (PANDAS, LOGGING, SMTPLIB, ETC.)

Python constitue le cœur du moteur d'analyse. Grâce à sa souplesse et à la richesse de son écosystème, il a permis l'implémentation de scripts de traitement automatisé des journaux d'accès.

- La bibliothèque « **pandas** » est utilisée pour charger, manipuler et analyser les données extraites des tables MySQL. Elle facilite l'identification des comportements suspects comme les accès hors horaires ou en masse.
- Le module « **logging** » permet de gérer la journalisation locale des événements et erreurs sur le système. Il est utile aussi bien pour le débogage que pour la traçabilité du fonctionnement du système.
- Le module « **configparser** » permet de stocker les paramètres de configuration (comme les identifiants de connexion à la base de données, les seuils de détection ou les horaires autorisés) dans un fichier « **.ini** », facilitant ainsi l'administration du système.
- Le module « **smtplib** » associé aux modules « **email.mime** » permet l'envoi automatique d'alertes par e-mail, notamment en cas d'anomalie détectée (ex. : accès non autorisé, consultation massive, etc.) [7]. Ces alertes peuvent contenir des pièces jointes comme des rapports CSV générés dynamiquement à l'aide de pandas.
- Le module « **os** » sert à créer le dossier exports si nécessaire, gérer les chemins et vérifier si des fichiers existent.



- Le module « **datetime** » sert à analyser l'heure d'une requête, calculer le délai entre deux requêtes ainsi que générer des noms de fichier avec l'horodatage

```
import os
import logging
import configparser
from datetime import datetime, timedelta, date
from collections import defaultdict
from email.mime.multipart import MIMEMultipart
from email.mime.base import MIMEBase
from email import encoders
import smtplib
import pandas as pd
import mysql.connector
```

Ces outils permettent de construire une analyse intelligente et automatisée du comportement des utilisateurs vis-à-vis des données sensibles.

2.4.3 FLASK POUR L'INTERFACE WEB

Pour rendre le système accessible aux utilisateurs (administrateurs, superviseurs de sécurité...), une interface web légère a été développée avec « Flask », un microframework web Python.

```
from flask import Flask, render_template, request, redirect, url_for, flash, Blueprint
import subprocess, shlex, tempfile, os, sys
import pandas as pd
import mysql.connector, configparser
```

- Le module « **Flask** » sert à créer et gérer une application web
- Le module « **render_template** » sert à afficher des pages HTML avec des données Python
- Le module « **request** » sert à lire les données envoyées depuis un formulaire
- Les modules « **redirect** », « **url_for** » sert à Naviguer entre les pages
- Le module « **flash** » sert à montrer des messages à l'utilisateur
- Le module « **Blueprint** » sert à structurer l'application en modules
- Le module « **subprocess** » sert à lancer des commandes système
- Le module « **shlex** » sert à structurer une commande shell et la découper proprement
- Le module « **tempfile** » sert à créer des fichiers et des dossiers temporaires
- Le module « **sys** » sert à contrôler le comportement global de Python



Cette interface permet de :

- Lancer manuellement des analyses de logs,
- Visualiser les anomalies détectées,
- Consulter les journaux d'accès récents,
- Gérer les paramètres de détection et les horaires d'accès,
- Télécharger les rapports générés.

L'utilisation de Flask permet une intégration fluide entre le backend Python/MySQL et le frontend HTML/CSS, tout en maintenant une architecture simple et efficace, adaptée à un usage interne dans un environnement hospitalier sécurisé.

2.4.4 SQLMAP – OUTIL DE TEST DE VULNERABILITES SQL

SQLMap est un outil open-source puissant permettant d'automatiser la détection et l'exploitation des injections SQL. Dans le cadre de ce projet, il a été utilisé pour simuler des attaques sur les paramètres vulnérables de certaines URLs et observer comment des requêtes malveillantes pouvaient être injectées dans la base de données [2].

Grâce à SQLMap, il a été possible de :

- Identifier les failles potentielles de type SQLi,
- Évaluer la robustesse de l'authentification et des requêtes SQL utilisées,
- Générer des logs utiles pour tester la capacité de détection d'anomalies de l'outil développé.

Cela a permis de valider l'efficacité du système d'analyse en identifiant et journalisant ces tentatives d'accès malveillants.

2.4.5 GIT – OUTIL DE GESTION DE VERSIONS

Git est un système de gestion de version décentralisé, très utilisé dans le développement logiciel. Bien que son usage dans ce projet n'ait pas été central, il aurait pu être utilisé pour :

- Conserver un historique clair des modifications du code,
- Faciliter le travail collaboratif (dans un environnement multi-développeurs),
- Assurer un retour en arrière rapide en cas d'erreurs.
- Son intégration via GitHub a amélioré la traçabilité et le partage du code.



2.4.6 HTML5 – CREATION DES PAGES WEB DE L'APPLICATION

HTML5 est le langage de base pour structurer les pages web. Il a été utilisé pour créer l'interface utilisateur de l'application Flask, notamment dans les fichiers :

- **base.html** : page d'accueil,
- **scan.html** : formulaire de lancement de l'analyse,
- **anomalie.html** : affichage des anomalies détectées,
- **dashboard.html** : vue d'ensemble du système et des statistiques.

HTML5 a permis de définir des structures claires et accessibles, telles que :

- Des formulaires de soumission,
- Des tableaux de données,
- Des liens de navigation.



3. TRAVAUX REALISES

Dans cette partie, je présente les étapes concrètes de mise en œuvre du projet. L'objectif était de bâtir un environnement hospitalier numérique sécurisé, capable de détecter et enregistrer toute activité anormale liée à l'accès aux données médicales. Pour cela, j'ai conçu une base de données relationnelle complète, intégrant à la fois la gestion médicale classique et les mécanismes de journalisation et de cybersécurité.

3.1 MISE EN PLACE DE LA BASE DE DONNEES SECURISEE

La première étape a consisté à créer une base de données robuste, adaptée au fonctionnement réel d'un hôpital. Elle regroupe à la fois les informations médicales (patients, soins, prescriptions, hospitalisations...) et les éléments de sécurité (utilisateurs, logs, anomalies). Le modèle relationnel repose sur des clés primaires/étrangères, des types bien définis, et des contraintes d'intégrité pour garantir la cohérence globale des données.

Tables :

- departements
- examens
- utilisateurs
- anomalies_detectees
- access_logs
- infirmieres
- prescriptions
- log_requetes
- consultations
- patients
- medecins
- hospitalisations
- medicaments

Vues :

- vue_patients_hospitalises
- vue_consultations_du_jour
- vue_historique_hospitalisations

Index :

- idx_patients_cin
- idx_consultations_date
- idx_hospitalisations_dates

Triggers :



- trg_log_insert_patients
- trg_log_update_patients
- trg_log_delete_patients
- trg_log_select_patients

3.1.1 TABLES PRINCIPALES (PATIENTS, MEDECINS, CONSULTATIONS, ETC.)

Les **tables métiers** couvrent l'essentiel des activités médicales et administratives :

- « **Patients** » contient les informations personnelles, médicales et d'urgence.
- « **Médecins** », « **infirmières** » et « **départements** » permettent de structurer les ressources humaines et les services de l'hôpital.
- « **Consultations** », « **prescriptions** », « **hospitalisations** », « **examens** », et « **médicaments** » assurent le suivi des soins, traitements et diagnostics.
- Les relations entre les entités sont définies avec précision

Cette architecture permet d'avoir une vision centralisée, claire et structurée du parcours de soins des patients.

3.1.2 TABLES DE JOURNALISATION ET ANOMALIES

Pour garantir la traçabilité des actions et la surveillance du système, plusieurs tables spécialisées ont été créées :

- **access_logs** : enregistre les accès bruts à toutes les tables critiques (qui, quand, où, quoi), avec l'adresse IP, le type d'opération (SELECT, DELETE...), et le statut de l'accès (autorisé ou refusé).
- **log_requetes** : stocke des informations plus détaillées à propos des requêtes SQL effectuées par les utilisateurs, avec leurs rôles, succès/échec, et commentaire éventuel.
- **anomalies_detectees** : centralise les activités jugées suspectes ou non conformes, avec une classification par gravité (faible à critique), un état de traitement, et une description libre.

Ces données permettent de reconstituer le comportement d'un utilisateur, de détecter des abus de privilèges, ou de repérer des accès à des moments ou à des zones sensibles.

3.1.3 INDEX, VUES ET TRIGGERS POUR LA SURVEILLANCE

Afin de faciliter la détection automatique d'anomalies et d'optimiser les performances, plusieurs mécanismes avancés ont été mis en place :

- **Index** : appliqués sur les colonnes critiques comme horodatage, utilisateur, ou type_operation, ils accélèrent les recherches dans les tables de logs.



- **Vues SQL** : créées pour générer des tableaux de bord synthétiques, comme les consultations par médecin, les opérations suspectes par jour, ou les utilisateurs ayant dépassé un seuil d'accès.
- **Triggers** : utilisés pour automatiser certaines actions de sécurité. Par exemple, dès qu'un utilisateur accède à une table sensible, un trigger ajoute une entrée dans access_logs ou log_requetes.

Table 1 : Vues

Nom de la vue	Description rapide
vue_patients_hospitalises	Liste des patients hospitalisés actuellement
vue_consultations_du_jour	Consultations médicales prévues pour aujourd'hui
vue_historique_hospitalisations	Historique complet des hospitalisations avec durée de séjour

Table 2 : Index

Nom de l'index	Description rapide
idx_patients_cin	Index sur la CIN des patients (accès rapide)
idx_consultations_date	Index sur la date des consultations
idx_hospitalisations_dates	Index sur les dates d'entrée/sortie des hospitalisations

Table 3 : Triggers

Nom du trigger	Moment	Description rapide
trg_log_insert_patients	AFTER INSERT	Log automatique après ajout d'un patient
trg_log_update_patients	AFTER UPDATE	Log après modification des infos patient
trg_log_delete_patients	AFTER DELETE	Log après suppression d'un patient
trg_log_select_patients	AFTER SELECT	Log après lecture de la table patients

Ce dispositif permet non seulement d'améliorer la réactivité du système, mais aussi de renforcer la **défense en profondeur**, en détectant les comportements anormaux dès leur apparition.



3.1.4 RELATIONS AVEC CARDINALITES DES ENTITES :

A. medecins — appartient → départements

- Relation : "appartient"
- Cardinalité : Un médecin appartient à 1 département, un département peut avoir 0 à N médecins
- (0,N) medecins — appartient → (1,1) departements

B. infirmieres — appartient → départements

- (0,N) infirmieres — appartient → (1,1) departements

C. utilisateurs — est lié à → medecins/infirmieres

- Un utilisateur peut représenter un médecin ou une infirmière
- (0,1) utilisateurs — est_medecin → (1,1) medecins
- (0,1) utilisateurs — est_infirmiere → (1,1) infirmieres

D. consultations — concerne → patients

- Un patient peut avoir plusieurs consultations
- (0,N) consultations — concerne → (1,1) patients

E. consultations — est faite par → medecins

- Un médecin peut faire plusieurs consultations
- (0,N) consultations — est_faite_par → (1,1) medecins

F. prescriptions — appartient à → consultations

- Une prescription est liée à une seule consultation
- (0,N) prescriptions — concerne → (1,1) consultations

G. prescriptions — prescrit → medicaments

- Une prescription fait référence à un seul médicament
- (0,N) prescriptions — prescrit → (1,1) medicaments

H. hospitalisations — concerne → patients

- Un patient peut être hospitalisé plusieurs fois
- (0,N) hospitalisations — concerne $\rightarrow$ (1,1) patients

I. hospitalisations — surveillée par $\rightarrow$ medecins

- Un médecin peut superviser plusieurs hospitalisations
- (0,N) hospitalisations — surveillée_par $\rightarrow$ (1,1) medecins

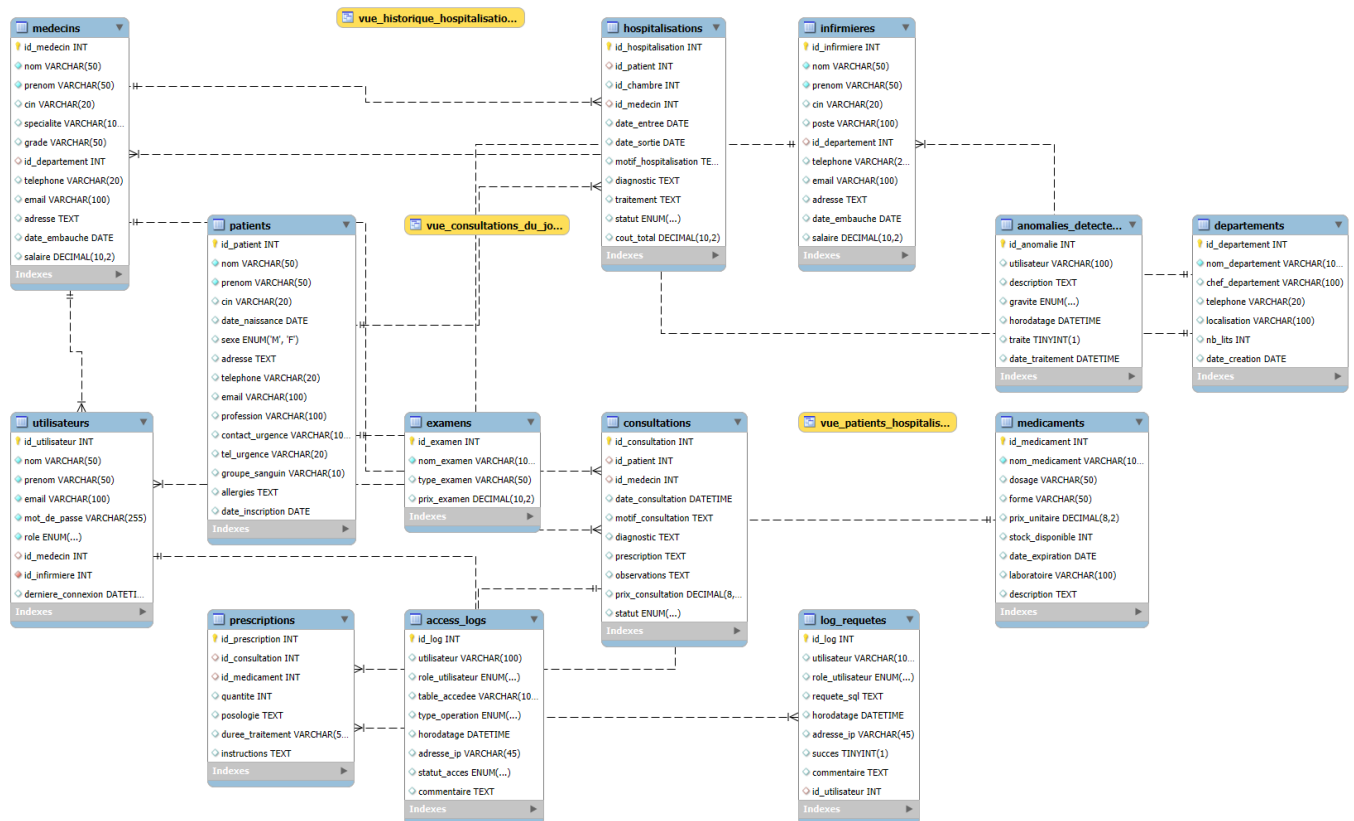
J. log_requetes — généré par $\rightarrow$ utilisateurs

- Un utilisateur peut générer plusieurs logs de requêtes
- (0,N) log_requetes — généré_par $\rightarrow$ (1,1) utilisateurs

K. anomalies_detectees — concerne $\rightarrow$ utilisateurs

- Une anomalie est liée à un utilisateur
- (0,N) anomalies_detectees — concerne $\rightarrow$ (1,1) utilisateurs

3.1.5 DIAGRAMME MCD :





3.1.6 LA CONNEXION A LA BASE DE DONNEES MYSQL :

```
DB_CFG = config["mysql"]

def get_connection(buffered: bool = False):
    return mysql.connector.connect(
        host=DB_CFG["host"],
        port=int(DB_CFG["port"]),
        user=DB_CFG["user"],
        password=DB_CFG["password"],
        database=DB_CFG["database"],
        buffered=buffered,
    )
```

Analyse.py

```
≡ config.ini
1  [mysql]
2  host = localhost
3  port = 3306
4  user = root
5  password = abirmajdi@2004
6  database = hopital_db
7
```

Config.ini

La fonction **get_connection** permet de se connecter automatiquement à la base de données MySQL utilisée dans le projet.

Elle utilise les informations de connexion (comme l'host, le mot de passe, le nom de la base, le port...) qui sont stockées dans config.ini (DB_CFG).

Cette fonction sera aussi utilisée dans app.py pour assurer que la connexion sera faite correctement.

3.2 DEVELOPPEMENT DE L'OUTIL D'ANALYSE PYTHON

Un outil Python personnalisé a été développé dans le cadre de ce projet pour surveiller les accès aux bases de données médicales sans nécessité de surveillance pratique constante. C'est une application préventive dont l'objectif est de prévenir les intrusions et de garantir la sécurité des informations sensibles sur les patients et les employés de l'hôpital. En particulier, l'outil cherche à être simple à utiliser, capable de réaliser des analyses sans intervention humaine et de rapporter les résultats dès qu'une anomalie est découverte.

3.2.1 JOURNALISATION

```
logger = logging.getLogger(__name__)

logger.setLevel(logging.INFO)

_handler = logging.FileHandler("analyseur.log", mode="a", encoding="utf-8")
_handler.setFormatter(
    logging.Formatter("%(asctime)s - %(levelname)s - %(message)s")
)
logger.handlers = []
logger.addHandler(_handler)
```

Ce code sert à créer un fichier « analyseur.log » où seront enregistrés automatiquement tous les événements importants qui se produisent pendant que le programme tourne (par exemple : une anomalie détectée, un e-mail envoyé, une erreur, etc.).

C'est très utile pour :

- Faire un suivi clair des actions du système,
- Comprendre ce qui s'est passé en cas de problème,
- Aider à la maintenance et au débogage.

3.2.2 ANALYSE DES LOGS ET DETECTION D'ANOMALIES

L'une des composantes principales du code développé est l'analyse automatique des fichiers de logs (access_logs, log_requetes). En outre, grâce à l'analyse périodique des journaux d'activité, les comportements suspects ou inhabituels, tels que des requêtes SQL, des accès en dehors des heures normales et des consultations injustifiées des dossiers médicaux, peuvent être rapidement détectés.

En général, l'algorithme de détection est basé sur une sorte de combinaison de règles statiques et de modèles comportementaux simples créés sur la base des habitudes des utilisateurs confirmés. Toutes les opérations effectuées dans la base de données sont strictement horodatées et sont liées à l'utilisateur effectuant l'interaction concernée.

a) EXPLICATION DE LA FONCTION « DETECTER_ET_STOCKER » :

Cette fonction regroupe les principales fonctionnalités de code « analyse.py ». Elle a pour mission :

- Garder uniquement les anomalies du jour pour insertion
- Analyser les logs, insère les anomalies du jour dans « anomalies_detectees »
- Charger les logs et connexion à la base de données
- Vérifier les colonnes essentielles



- Détection des anomalies comme accès hors horaires, requêtes répétées sur consultation, tentatives refusées multiples et finalement tentatives échouées multiples

```
def detecter_et_stocker():
    # Analyse les logs, insère les anomalies du jour dans anomalies_detectees
    conn = get_connection()
    cursor = conn.cursor(dictionary=True)
    logger.info("Connexion MySQL réussie.")

    # Charger les logs
    cursor.execute("SELECT * FROM log_requetes")
    df_log = pd.DataFrame(cursor.fetchall(), columns=[c[0].lower() for c in cursor.description])
    logger.info("%d lignes log_requetes chargées.", len(df_log))

    cursor.execute("SELECT * FROM access_logs")
    df_access = pd.DataFrame(cursor.fetchall(), columns=[c[0].lower() for c in cursor.description])

    logger.info("%d lignes access_logs chargées.", len(df_access))
```

Exemple de l'output :

```
2025-07-19 16:10:43,049 - INFO - Connexion MySQL réussie.
2025-07-19 16:10:43,060 - INFO - 17 lignes log_requetes chargées.
2025-07-19 16:10:43,065 - INFO - 12 lignes access_logs chargées.
```

Analyseur.log

b) CONNEXION A LA BASE DE DONNEES ET CHARGEMENT DES LOGS :

La première partie de cette fonction cherche à établir la connexion à la base de données de type MySQL et charger les logs. Par suite deux types de données sont ensuite appelés dans cette étape comme :

- log_requetes : les requêtes reçues dans le système
- access_logs : les logs d'accès aux données médicales

Ces données sont ensuite transformées en pandas DataFrame pour pouvoir les analyser

c) VERIFICATION DE LA QUALITE DES DONNEES :

```
# Vérifier les colonnes essentielles
def check_cols(df: pd.DataFrame, cols: list[str]):
    for col in cols:
        if col not in df.columns:
```



```
raise ValueError(f"Colonne manquante : {col}")
if df[col].isnull().all():
    logger.warning("Colonne %s entièrement vide", col)

check_cols(df_log, ["horodatage"])
check_cols(df_access, ["horodatage"])
```

La fonction interne check_cols garantit que les colonnes essentielles comme horodatage ne sont pas vides et interdit les colonnes incomplètes

d) DETERMINATION DYNAMIQUE DES COLONNES UTILISATEURS

```
# Colonnes utilisateur dynamiques
user_col_log = "id_utilisateur" if "id_utilisateur" in df_log.columns else
"utilisateur"
user_col_acc = "utilisateur" if "utilisateur" in df_access.columns else
"id_utilisateur"
```

Les table **acces_log** et **log_requetes** contient le champ « utilisateur » alors que la table **utilisateurs** contient le champ « id_utilisateurs ». Alors cette partie assure la compatibilité le champ appelé « utilisateur » ou « id_utilisateur » peut être adaptable

e) DICTIONNAIRE ID UTILISATEUR → ADRESSE E-MAIL

```
# Dictionnaire id->email
cursor.execute("SELECT id_utilisateur, email FROM utilisateurs")
dict_id_to_email = {
    row["id_utilisateur"]: row["email"] for row in cursor.fetchall()
}
anomalies: list[dict] = []
AUJ = date.today()
```

Ce dictionnaire sert à retrouver le nom ou l'e-mail de l'utilisateur responsable de l'anomalie, pour l'enregistrer dans la table « anomalies_detectees » puis dans le contenu du rapport/alerte et garder uniquement les anomalies du jour meme du scan pour l'insertion

f) DETECTION DES ANOMALIES

1er) ACCES EN DEHORS DES HORAIRES AUTORISES (22H–6H) :

```
# — Accès hors horaires (22h-6h)
for _, row in df_log.iterrows():
    ts = row["horodatage"]
    if pd.notnull(ts) and (ts.hour >= 22 or ts.hour < 6):
        anomalies.append(
            {
```



```

        "utilisateur": row.get(user_col_log),
        "horodatage": ts,
        "type_incident": "Accès hors horaires",
        "description": f"Requête à {ts}",
        "gravite": "moyenne",
    }
)
for _, row in df_access.iterrows():
    ts = row["horodatage"]
    if pd.notnull(ts) and (ts.hour >= 22 or ts.hour < 6):
        anomalies.append(
            {
                "utilisateur": row.get(user_col_acc),
                "horodatage": ts,
                "type_incident": "Accès hors horaires (access_logs)",
                "description": f"Accès {row.get('table_accedee', 'inconnu')}
à {ts}",
                "gravite": "moyenne",
            }
        )

```

Toute tentative d'accès dehors des horaires autorisés (22h–6h) sera marqué comme suspecte et par conséquent une anomalie sera générée avec la date et l'heure pour chaque cas détecté avec une gravité moyenne.

2e) REQUETES REPETEES SUR "CONSULTATION" :

```

# — Requetes répétées (10 en ≤ 10 min) sur consultations
df_consult = df_log[df_log.get("commentaire",
    "").str.lower().str.contains("consultation", na=False)]
user_times: dict = defaultdict(list)
for _, row in df_consult.iterrows():
    uid, ts = row.get(user_col_log), row["horodatage"]
    if pd.notnull(uid) and pd.notnull(ts):
        user_times[uid].append(ts)
for uid, times in user_times.items():
    times.sort()
    for i in range(len(times) - 10):
        if times[i + 10] - times[i] <= timedelta(minutes=10):
            anomalies.append(
                {
                    "utilisateur": uid,
                    "horodatage": times[i + 10],
                    "type_incident": "Requetes répétées",
                    "description": f"10+ requêtes entre {times[i]} et
{times[i+10]}",
                    "gravite": "élevée", })

```



break

Si un utilisateur exécute plus de 10 requêtes sur consultation en 10 minutes, cela peut révéler un script ou une extraction de masse. Cette situation est classée dans le script comme une anomalie à haute gravité.

3e) TENTATIVES D'ACCES REFUSEES (ACCESS_LOGS) :

```
# — Tentatives refusées multiples (access_logs → statut_acces = refuse)
if "statut_acces" in df_access.columns:
    df_refuse = df_access[df_access["statut_acces"].str.lower() == "refuse"]
    fails: dict = defaultdict(list)
    for _, row in df_refuse.iterrows():
        uid, ts = row.get(user_col_acc), row["horodatage"]
        if pd.notnull(uid) and pd.notnull(ts):
            fails[uid].append(ts)
    for uid, times in fails.items():
        times.sort()
        for i in range(len(times) - 4):
            if times[i + 4] - times[i] <= timedelta(minutes=5):
                anomalies.append(
                    {
                        "utilisateur": uid,
                        "horodatage": times[i + 4],
                        "type_incident": "Tentatives refusées multiples",
                        "description": f"5 échecs entre {times[i]} et {times[i+4]}",
                        "gravite": "élevée",
                    }
                )
        break
```

Lorsqu'un utilisateur n'arrive pas à se connecter 5 fois en moins de 5 minutes cela peut être interpréter comme attaque par force brute et par la suite une anomalie de gravité élevée sera générée.

4e) REQUETES ECHOUÉES (LOG_REQUETES) :

```
# — Tentatives échouées multiples (log_requetes → succes = False)
if "succes" in df_log.columns:
    df_fail = df_log[df_log["succes"] == False]
    fails: dict = defaultdict(list)
    for _, row in df_fail.iterrows():
        uid, ts = row.get(user_col_log), row["horodatage"]
        if pd.notnull(uid) and pd.notnull(ts):
            fails[uid].append(ts)
```



```

for uid, times in fails.items():
    times.sort()
    for i in range(len(times) - 4):
        if times[i + 4] - times[i] <= timedelta(minutes=5):
            anomalies.append(
                {
                    "utilisateur": uid,
                    "horodatage": times[i + 4],
                    "type_incident": "Tentatives échouées multiples",
                    "description": f"5 échecs entre {times[i]} et {times[i+4]}",
                    "gravite": "moyenne",
                }
            )
    break

```

Un utilisateur qui effectue des requêtes et qui échoue 5 fois en 5 minutes, en particulier lorsque cela se produit fréquemment de manière simultanée pour accroître la menace, peut signifier des tentatives d'injection ou d'exploration illégitime. Une anomalie de gravité moyenne est donc générée.

3.2.3 ENREGISTREMENT DES ANOMALIES DETECTEES

```

# Garder uniquement les anomalies du jour pour insertion
anomalies_today = [a for a in anomalies if a["horodatage"].date() == AUJ]
logger.info("%d anomalies détectées aujourd'hui.", len(anomalies_today))

```

Cette ligne garde uniquement les anomalies dont la date correspond à aujourd'hui (AUJ) qui seront inséré dans la base

Une ligne de journal (logger.info) affiche combien d'anomalies ont été trouvées aujourd'hui.

```
2025-07-15 01:04:51,482 - INFO - 6 anomalies détectées aujourd'hui.
```

Analyseur.log

Dès que cela se produit, une anomalie est ajoutée automatiquement à la table « **anomalies_detectees** ». Chaque entrée comprend le nom ou l'ID de l'utilisateur ou l'email à l'origine du comportement anormal. Elle ajoute quelques détails, notamment la nature de l'anomalie, le type d'accès, la date et l'heure de l'incident, le niveau de gravité et la table sur laquelle la violation s'est produite ainsi que la gravité de ce comportement.

```

# Insertion dans la table
insert_sql = """
    INSERT INTO anomalies_detectees (utilisateur, description, gravite,
    horodatage)

```



```
VALUES (%s, %s, %s, %s)
"""
for a in anomalies_today:
    uid = a["utilisateur"]
    email = dict_id_to_email.get(uid) if isinstance(uid, int) else None
    utilisateur = email if email else str(uid)
    cursor.execute(insert_sql, (
        utilisateur,
        a["description"],
        a["gravite"],
        a["horodatage"],
    ))
```

Une fois enregistrées, ces données seront utiles pour mettre au point les règles de détection des anomalies. En plus de cela, une visibilité améliorée des anomalies sera offerte aux administrateurs de la base de données.

Le fait d'automatiser l'enregistrement permet non seulement un gain de temps considérable, mais garantit aussi que rien n'échappe à la surveillance du système, même en dehors des heures de présence des équipes techniques.

```
conn.commit()
cursor.close()
conn.close()
```

Ces 3 lignes servent à garantir la validation et fermeture de la connexion

3.2.4 GENERATION DE RAPPORTS CSV

Afin de faciliter l'audit et la remontée d'informations, l'outil génère régulièrement des rapports CSV détaillés contenant toutes les anomalies détectées. Ces rapports peuvent être triés par date, utilisateur ou type d'anomalie. Ils sont stockés dans un répertoire dédié et peuvent être exploités à la fois par l'équipe informatique pour investiguer davantage, et par les responsables de la conformité pour assurer la sécurité réglementaire des systèmes d'information de l'hôpital.

Découvrant la partie responsable de la génération des rapports CSV pour mieux comprendre

```
def generer_rapport_et_envoyer():
    conn = get_connection()
    cursor = conn.cursor(dictionary=True)
```

- La fonction « generer_rapport_et_envoyer » a pour but de générer les rapports et envoyer un email en faisant appel à une autre fonction qu'on verra par la suite.



- En utilisant la fonction `get_connection()` la connexion à la base MySQL sera établit avec un curseur configuré avec `dictionary=True` pour que les résultats soient des dictionnaires.

```
cursor.execute(
    """
    SELECT utilisateur, description, gravite, horodatage
    FROM anomalies_detectees
    WHERE DATE(horodatage) = CURDATE()
    ORDER BY horodatage DESC
    """ )
rows = cursor.fetchall()
cursor.close()
conn.close()
```

Cette partie a pour but de :

- Récupérer toutes les anomalies détectées le jour du scan.
- Trier les résultats par horodatage décroissant (les plus récentes en premier).
- Fermer proprement la connexion à la base.

```
if not rows:
    logger.warning("Aucune anomalie enregistrée aujourd'hui dans anomalies_detectees.")
    return
```

Ensuite on vérifie si aucune anomalie est enregistrée aujourd'hui si oui un message d'avertissement sera enregistré et affiché dans le fichier « analyseur.log » et la fonction s'arrête ici.

Exemple de l'output :

```
2025-07-19 16:10:43,111 - WARNING - Aucune anomalie enregistrée aujourd'hui dans anomalies_detectees.
```

Analyseur.log

```
df = pd.DataFrame(rows)
os.makedirs("exports", exist_ok=True)
fname = f"exports/rapport_incidents_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv"
df.to_csv(fname, index=False, encoding="utf-8-sig")
logger.info("%d anomalies du jour exportées dans %s", len(df), fname)
```

- Un dossier exports est créé s'il n'existe pas.
- Un fichier CSV est généré avec un nom basé sur la date et l'heure actuelles.

- Un message est enregistré dans « analyseur.log » contenant le nombre d'anomalies exportées et le chemin du fichier CSV généré.

Exemple de l'output :

```
2025-07-15 01:04:51,510 - INFO - 16 anomalies du jour exportées dans
exports/rapport_incidents_20250715_010451.csv
```

Analyseur.log

```
exports > ■ rapport_incidents_20250715_010451.csv
1  utilisateur,description,gravite,horodatage
2  test.user,Accès à patients à 2025-07-15 00:54:32,moyenne,2025-07-15 01:01:43
3  test.user,Accès à patients à 2025-07-15 00:55:02,moyenne,2025-07-15 01:01:43
4  test.user,Accès à patients à 2025-07-15 00:55:32,moyenne,2025-07-15 01:01:43
5  test.user,Accès à patients à 2025-07-15 00:56:02,moyenne,2025-07-15 01:01:43
6  test.user,Accès à patients à 2025-07-15 00:56:32,moyenne,2025-07-15 01:01:43
7  test.user,5 échecs entre 2025-07-15 00:54:32 et 2025-07-15 00:56:32,moyenne,2025-07-15 01:01:43
8  test.user,Accès patients à 2025-07-15 00:56:32,moyenne,2025-07-15 00:56:32
9  test.user,5 échecs entre 2025-07-15 00:54:32 et 2025-07-15 00:56:32,moyenne,2025-07-15 00:56:32
10 test.user,Accès patients à 2025-07-15 00:56:02,moyenne,2025-07-15 00:56:02
11 test.user,Accès patients à 2025-07-15 00:55:32,moyenne,2025-07-15 00:55:32
12 test.user,Accès patients à 2025-07-15 00:55:02,moyenne,2025-07-15 00:55:02
13 test.user,Accès patients à 2025-07-15 00:54:32,moyenne,2025-07-15 00:54:32
14 dr.ahmed,Accès à patients à 2024-07-06 03:42:00,moyenne,2025-07-15 00:36:04
15 infirmiere.rania,Accès à consultations à 2024-07-06 04:15:00,moyenne,2025-07-15 00:36:04
16 test.user,Accès à hospitalisations à 2025-07-13 02:45:00,moyenne,2025-07-15 00:36:04
17 rania.khadiri@chu.ma,10+ requêtes entre 2025-07-13 10:00:00 et 2025-07-13 10:09:30,elevation,2025-07-15 00:36:04
18
```

rapport_incidents_20250715_010451.csv

Cette fonctionnalité permet également de préparer des rapports CSV contenant les anomalies détecté le jour même destiné au Responsable du système d'information (RSI), avec des chiffres clairs sur le niveau de sécurité du système d'information (SI) hospitalier.

3.2.5 ENVOI D'E-MAILS D'ALERTE AUTOMATIQUE

Un module d'envoi d'alertes par e-mail en temps réel est également intégré dans le système, ce qui garantit qu'une réactivité maximale peut être obtenue. L'e-mail avec l'incident critique, tel qu'un accès suspect à la base de données des patients sous un compte d'utilisateur ou à 3 heures du matin, est envoyé au RSI dès que l'incident est découvert.

L'e-mail contient la nature de l'incident, la date, l'heure, l'ID de l'utilisateur, la description de ce qui s'est passé et le niveau de gravité. La détection automatisée signifie que le RSI peut répondre rapidement et augmenter la vigilance sur l'ensemble du système avant que des dommages plus importants ne soient causés.

Représentant une explication claire de la partie du code responsable de l'envoi d'e-mails d'alerte automatique :

```
def envoyer_email(chemin_fichier: str)
```

C'est la fonction utilisée pour l'envoi d'un e-mail contenant en pièce jointe un rapport CSV d'anomalies avec le chemin en argument

1. CREATION DE L'E-MAIL :

```
try:
    msg = MIMEMultipart()
    msg["From"] = MAIL_CFG["from"]
    msg["To"] = MAIL_CFG["to"]
    msg["Subject"] = "Rapport quotidien des anomalies détectées"
```

Cette partie permet de :

- Créer un message avec pièce jointe
- Le sujet est fixe : "Rapport quotidien des anomalies détectées"
- Récupérer L'expéditeur et le destinataire depuis MAIL_CFG

```
MAIL_CFG = config["email"]
```

```
[email]
from = p02009187@gmail.com
to = saidmajdi123@gmail.com
smtp = smtp.gmail.com
port = 587
password = xthy nsrt zuft jtwv
```

Config.ini

2. AJOUT DU FICHIER CSV EN PIECE JOINTE :

```
with open(chemin_fichier, "rb") as f:
    part = MIMEBase("application", "octet-stream")
    part.set_payload(f.read())
    encoders.encode_base64(part)
    part.add_header(
        "Content-Disposition",
        f"attachment; filename={os.path.basename(chemin_fichier)}",
    )
    msg.attach(part)
```

- Le fichier CSV est ouvert en mode binaire (rb).
- Il est converti en flux attachable (MIMEBase) et encodé en Base64 (standard pour les mails).
- On lui donne un nom lisible



- On attache ce fichier à l'e-mail.

Exemple de l'output :

```
2025-07-15 01:04:51,536 - INFO - Tentative d'envoi du mail à
saidmajdi123@gmail.com
2025-07-15 01:04:53,003 - INFO - E-mail envoyé avec succès.
```

Analyseur.log

3. ENVOI VIA UN SERVEUR SMTP SECURISE :

```
logger.info("Tentative d'envoi du mail à %s", MAIL_CFG["to"])
    with smtplib.SMTP(MAIL_CFG["smtp"], int(MAIL_CFG["port"])) as server:
        server.starttls()
        server.login(MAIL_CFG["from"], MAIL_CFG["password"])
        server.send_message(msg)
    logger.info("E-mail envoyé avec succès.")
```

Cette partie assure :

- Connexion au serveur SMTP défini dans MAIL_CFG (smtp.gmail.com).
- Une communication sécurisée.
- Connexion avec l'identifiant et mot de passe.
- Envoi du message complet.
- Affichage du message « E-mail envoyé avec succès » dans « analyseur.log »

4. GESTION D'ERREUR

```
except Exception:
    logger.exception("Échec lors de l'envoi de l'e-mail :")
```

Si une erreur survient à n'importe quelle étape, elle sera enregistrée dans « analyseur.log » avec tous les détails.

Exemple de l'output :

```
2025-07-11 19:08:45,918 - ERROR - Erreur lors de l'envoi de l'e-mail : (535,
b'5.7.8 Username and Password not accepted. For more information, go
to\n5.7.8 https://support.google.com/mail/?p=BadCredentials ffacd0b85a97d-
3b5e8e1e3bdsm5005165f8f.81 - gsmtpt')
```

Analyseur.log



Rapport quotidien des anomalies détectées



Boîte de réception x



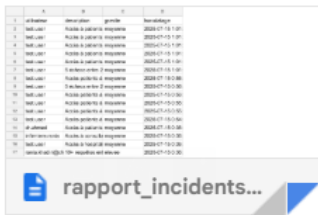
p02009187@gmail.com

mar. 15 juil. 01:04 (il y a 6 jours)



À moi ▾

1 pièce jointe • Analyse effectuée par Gmail ⓘ



↳ Répondre

➞ Transférer



L'e-mail reçu

rapport_inc ... _010451.csv					Ouvrir avec ▾
	A	B	C	D	
1	utilisateur	description	gravite	horodatage	
2	test.user	Accès à patients à 2	moyenne	2025-07-15 1:01:43	
3	test.user	Accès à patients à 2	moyenne	2025-07-15 1:01:43	
4	test.user	Accès à patients à 2	moyenne	2025-07-15 1:01:43	
5	test.user	Accès à patients à 2	moyenne	2025-07-15 1:01:43	
6	test.user	Accès à patients à 2	moyenne	2025-07-15 1:01:43	
7	test.user	5 échecs entre 2025	moyenne	2025-07-15 1:01:43	
8	test.user	Accès patients à 202	moyenne	2025-07-15 0:56:32	
9	test.user	5 échecs entre 2025	moyenne	2025-07-15 0:56:32	
10	test.user	Accès patients à 202	moyenne	2025-07-15 0:56:02	
11	test.user	Accès patients à 202	moyenne	2025-07-15 0:55:32	
12	test.user	Accès patients à 202	moyenne	2025-07-15 0:55:02	
13	test.user	Accès patients à 202	moyenne	2025-07-15 0:54:32	
14	dr.ahmed	Accès à patients à 2	moyenne	2025-07-15 0:36:04	
15	infirmiere.rania	Accès à consultation	moyenne	2025-07-15 0:36:04	
16	test.user	Accès à hospitalisati	moyenne	2025-07-15 0:36:04	
17	rania.khadiri@chu.m	10+ requêtes entre 2	elevée	2025-07-15 0:36:04	

Le rapport reçu



3.3 DEPLOIEMENT DE L'APPLICATION WEB FLASK :

Ce travail a abouti à la création et au déploiement d'une application web basée sur le micro-framework Flask qui s'intègre à la base de données médicale en tout simplicité [5]. D'une part, cette application peut faire office de tableau de bord, d'un moteur analytique et un point d'entrée pour activer des tests de sécurité SQL sur les URL de l'infrastructure hospitalière.

Le fichier « app.py » contient toute la logique backend de l'application. Il établit la connexion à la base de données MySQL via un fichier de configuration externe « config.ini », gère les différentes routes Flask, et intègre à la fois des fonctions d'affichage, d'analyse et de détection de vulnérabilités.

3.3.1 INTERFACE UTILISATEUR ET AFFICHAGE DES DONNEES

L'interface utilisateur est composée de plusieurs pages dynamiques rendues par Flask grâce au système de templates (HTML). Ces pages html hérite d'un fichier base.html :

base.html – Modèle de base des pages :

```
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>{% block title %}CHU Cyber-Dashboard{% endblock %}</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="bg-light">
<nav class="navbar navbar-expand-lg navbar-dark bg-primary mb-4">
  <div class="container-fluid">
    <a class="navbar-brand" href="{{ url_for('dashboard') }}">Cyber-Hopital</a>
    <div class="collapse navbar-collapse">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item"><a class="nav-link" href="{{ url_for('dashboard') }}">Dashboard</a></li>
        <li class="nav-item"><a class="nav-link" href="{{ url_for('anomalies') }}">Anomalies</a></li>
        <li class="nav-item"><a class="nav-link" href="{{ url_for('scan') }}">SQLMap Scan</a></li>
      </ul>
    </div>
  </div>
</nav>
<div class="container">
```



```
{% with messages = get_flashed_messages(with_categories=true) %}
{% if messages %}
{% for cat, msg in messages %}
<div class="alert alert-{{cat}}">{{ msg }}</div>
{% endfor %}
{% endif %}
{% endwith %}
{% block content %}{% endblock %}
</div>
</body>
</html>
```

Ce fichier **base.html** sert de modèle commun à toutes tes pages. Il

- Fournit une structure HTML complète (entête, navigation).
- Affiche automatiquement les messages système (flash) par exemple : un message de succès après l'analyse ou une erreur SQL.
- Laisse un emplacement vide pour le contenu spécifique de chaque page par exemple : dashboard.html, scan.html, anomalies.html injecteront ici leurs tableaux, boutons, ou formulaires.
- Centralise le style et la structure pour que les autres pages soient plus simples et uniformes.

Et chaque page correspond à une route définie dans app.py :

a) TABLEAU DE BORD :

```
# Tableau de bord
@app.route("/")
def dashboard():
    conn = get_connection(buffered=True)
    cur = conn.cursor()
    cur.execute("SELECT COUNT(*) FROM patients")
    nb_patients = cur.fetchone()[0]
    cur.execute("SELECT COUNT(*) FROM anomalies_detectees")
    nb_anomalies = cur.fetchone()[0]
    cur.close(); conn.close()
    return render_template(
        "dashboard.html",
        nb_patients=nb_patients,
        nb_anomalies=nb_anomalies,
    )
```

/ : Route principale qui affiche un tableau de bord en affichant deux statistiques générales : nombre de patients et nombre d'anomalies



Le Code HTML correspondant **dashboard.html** :

```
{% extends "base.html" %}
{% block title %}Dashboard - Cyber-Hopital{% endblock %}
{% block content %}
<h1 class="mb-4">Vue d'ensemble</h1>

<div class="row">
  <div class="col-md-4">
    <div class="card text-center shadow-sm">
      <div class="card-body">
        <h5 class="card-title">Patients</h5>
        <p class="display-6">{{ nb_patients }}</p>
      </div>
    </div>
  </div>
  <div class="col-md-4">
    <div class="card text-center shadow-sm">
      <div class="card-body">
        <h5 class="card-title">Anomalies détectées</h5>
        <p class="display-6">{{ nb_anomalies }}</p>
      </div>
    </div>
  </div>
</div>

<!-- Bouton vers scan SQLMap -->
<a href="{{ url_for('scan') }}" class="btn btn-warning mt-4">Lancer un scan
SQLMap</a>

<!-- Formulaire pour lancer l'analyse -->
<form action="{{ url_for('analyse') }}" method="post" class="mt-2">
  <button class="btn btn-danger">Lancer une nouvelle analyse</button>
</form>
{% endblock %}
```

Ce fichier contient :

- Hérite de la structure de base.html
- Contient le titre de l'onglet du navigateur
- Affiche le nombre total de patients et le nombre d'anomalies détectées
- Contient un bouton "Scan SQLMap" qui lance un scan de vulnérabilité
- Contient un formulaire "Analyse" qui déclenche une nouvelle analyse de logs



```
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a pr
oduction deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.1.5:5000
Press CTRL+C to quit
```

Terminal app.py

On ouvre le lien « <http://127.0.0.1:5000> » dans Microsoft Edge et on observe le tableau de bord. On remarque la présence de :

- **Section principale** : Titre "Vue d'ensemble", ce qui suggère qu'on est sur une interface de suivi global des activités du système.
- **Statistiques visibles** :
 - **Nombre de patients** : 5
 - **Anomalies détectées** : 110
- **Actions possibles** :
 - Bouton jaune : "Lancer un scan SQLMap" — il s'agit d'un outil pour détecter des vulnérabilités SQL.
 - Bouton rouge : "Lancer une nouvelle analyse" — pour lancer une nouvelle analyse (appelle analyse.py)



Cyber-Hopital

Vue d'ensemble

Patients

5

Anomalies détectées

110

Lancer un scan SQLMap

Lancer une nouvelle analyse

b) LISTE DES ANOMALIES :

```
# Liste des anomalies
@app.route("/anomalies")
def anomalies():
    conn = get_connection()
    df = pd.read_sql(
        "SELECT * FROM anomalies_detectees ORDER BY horodatage DESC",
        conn,
    )
    conn.close()
    return render_template(
        "anomalies.html",
        tables=[df.to_html(classes="table table-sm", index=False)],
    )
```

/anomalies : Cette page Récupère toutes les anomalies depuis « anomalies_detectees » et les affiche dans un tableau HTML en les classant par date décroissante.

Le Code HTML correspondant **anomalies.html** :

```
{% extends "base.html" %}
{% block title %}Anomalies{% endblock %}
```

```
{% block content %}
<h1>Anomalies détectées</h1>
{{ tables|safe }}
{% endblock %}
```

Ce fichier HTML :

- Hérite du fichier base.html
- Définit un titre personnalisé
- Affiche une section principale avec un tableau HTML dynamique
- Insère du HTML généré en Python

Cyber-Hopital

Analyse terminée – consultez la liste des anomalies.

Anomalies détectées

id_anomalie	utilisateur	description	gravite	horodatage
99	test.user	Accès à patients à 2025-07-15 00:54:32	moyenne	2025-07-15 01:01:43
100	test.user	Accès à patients à 2025-07-15 00:55:02	moyenne	2025-07-15 01:01:43
101	test.user	Accès à patients à 2025-07-15 00:55:32	moyenne	2025-07-15 01:01:43
102	test.user	Accès à patients à 2025-07-15 00:56:02	moyenne	2025-07-15 01:01:43
103	test.user	Accès à patients à 2025-07-15 00:56:32	moyenne	2025-07-15 01:01:43
104	test.user	5 échecs entre 2025-07-15 00:54:32 et 2025-07-15 00:56:32	moyenne	2025-07-15 01:01:43

Exemple de la page anomalie



c) DETAILS D'UN PATIENT (SECURISE) :

```
@app.route("/patient")
def patient():
    id_patient = request.args.get("id", "")
    conn = get_connection()
    cursor = conn.cursor(dictionary=True)

    cursor.execute("SELECT * FROM patients WHERE id_patient = %s", (id_patient,))
    patient = cursor.fetchone()

    cursor.close()
    conn.close()

    if patient:
        return f"""
        <h1>Données patient pour ID {id_patient}</h1>
        <p>Nom: {patient['nom']}</p>
        <p>Prénom: {patient['prenom']}</p>
        """
    else:
        return f"<p>Aucun patient avec ID {id_patient}</p>"
```

/patient : cette route récupère les données d'un patient par son ID et affiche ses infos : nom et prénom

d) DETAILS D'UN PATIENT (NON SECURISE / VOLONTAIREMENT VULNERABLE) :

```
@app.route("/patient-insecure")
def patient_insecure():
    id_patient = request.args.get("id", "")
    conn = get_connection()
    cursor = conn.cursor(dictionary=True)

    # VULNÉRABLE À L'INJECTION SQL
    query = f"SELECT * FROM patients WHERE id_patient = {id_patient}"
    cursor.execute(query)
    patient = cursor.fetchone()
    cursor.close()
    conn.close()

    if patient:
        return f"""
        <h1>[INSECURE] Données patient pour ID {id_patient}</h1>
        <p>Nom: {patient['nom']}</p>
        <p>Prénom: {patient['prenom']}</p>
        """
```



```
else:
    return f"<p>.....Aucun patient avec ID {id_patient}</p>"
```

/patient-insecure : cette route fait la même chose que **/patient** mais elle est volontairement vulnérable à l'injection SQL, ce qui permet de tester les mécanismes de détection de failles.

```
query = f"SELECT * FROM patients WHERE id_patient = {id_patient}"
```

Permet de démontrer une faille SQL (ex : ?id=1 OR 1=1)

3.3.2 EXPORTATION DES DONNEES ET CONSULTATION DES LOGS

L'application web intègre également des fonctionnalités avancées pour la sécurité :

a) ENVOI D'ALERTE E-MAIL :

```
def send_alert_email(vuln_type, target_url):
    cfg = configparser.ConfigParser()
    cfg.read(CFG_FILE)

    msg = MIMEMultipart()
    msg['From'] = cfg["email"]["from"]
    msg['To'] = cfg["email"]["to"]
    msg['Subject'] = f"ALERTE SÉCURITÉ - SQL Injection Détectée - Niveau: MOYEN"

    body = f"""
ALERTE SÉCURITÉ SYSTÈME
=====

URL testée: {target_url}
Base de données: {cfg['mysql']['database']}
Niveau de risque: MOYEN
Date/Heure: {pd.Timestamp.now().strftime('%Y-%m-%d %H:%M:%S')}

VULNÉRABILITÉ DÉTECTÉE: {vuln_type}

Actions recommandées:
- Vérifier immédiatement la sécurité du site
- Corriger les vulnérabilités SQL
- Mettre à jour les protections

Ce message a été généré automatiquement par le système de détection.
"""

    msg.attach(MIMEText(body, 'plain'))

    with smtplib.SMTP(cfg["email"]["smtp"], int(cfg["email"]["port"])) as
server:
```



```
server.starttls()
server.login(cfg["email"]["from"], cfg["email"]["password"])
server.send_message(msg)
```

Ce code permet :

- Chargement de la configuration
- Création du message e-mail
- Message automatiquement généré avec :
 - L'URL testée par l'outil (ex: SQLMap)
 - Le type de vulnérabilité détectée (In-band, Blind, Out-of-band)
 - La base de données concernée
 - La date et l'heure
 - Des recommandations de sécurité
- Envoi sécurisé de l'e-mail
- Sécurité et bonnes pratiques présentes

Dès qu'une vulnérabilité SQL est détectée, une alerte détaillée est envoyée par e-mail, ce qui permet une réaction rapide et adaptée par l'équipe de sécurité.

b) LANCER UNE NOUVELLE ANALYSE :

```
# Lancer une nouvelle analyse (appelle analyse.py)
@app.route("/analyse", methods=["POST"])
def analyse():
    cmd = [sys.executable, ANALYSE_PY]
    res = subprocess.run(cmd, capture_output=True, text=True)
    if res.returncode == 0:
        flash("Analyse terminée - consultez la liste des anomalies.", "success")
    else:
        flash("Erreur pendant l'analyse:\n" + res.stderr, "danger")
    return redirect(url_for("anomalies"))
```

- Lance le script analyse.py en utilisant subprocess.
- Si l'analyse réussit, un message de succès s'affiche.
- Sinon, un message d'erreur avec la trace.

c) Scan de vulnérabilité (avec SQLMap)

```
@app.route("/scan", methods=["GET", "POST"])
def scan():
    if request.method == "POST":
        target = request.form["url"]
        with tempfile.TemporaryDirectory() as tmp:
            sqlmap_dir = r"C:\Users\saidm\Documents\sqlmap"
```



```

sqlmap_path = os.path.join(sqlmap_dir, "sqlmap.py")

sorties = []

sorties.append("Étape 1 : Test de vulnérabilité SQL sur l'URL
cible...")

cmd_test = f'"{sys.executable}" "{sqlmap_path}" -u
{shlex.quote(target)} --batch --risk=3 --level=5'
res_test = subprocess.run(shlex.split(cmd_test),
capture_output=True, text=True, timeout=300)

sorties.append("Résultat du test de vulnérabilité :\n" +
res_test.stdout + res_test.stderr)

# Analyse du résultat
if any(keyword in res_test.stdout.lower() for keyword in ["boolean-
based blind", "union query", "error-based", "time-based", "out-of-band"]):
    if "union query" in res_test.stdout.lower():
        vuln_type = "Injection SQL classique (In-band SQLi)"
    elif "out-of-band" in res_test.stdout.lower():
        vuln_type = "Injection SQL hors bande (Out-of-band SQLi)"
    elif "boolean-based blind" in res_test.stdout.lower():
        vuln_type = "Injection SQL aveugle (Blind SQLi)"
    else:
        vuln_type = "Type inconnu"

# Envoi d'email
send_alert_email(vuln_type, target)
sorties.append(f"Vulnérabilité détectée : {vuln_type}. Un e-
mail d'alerte a été envoyé.")

# Dump
cmd_dump = f'"{sys.executable}" "{sqlmap_path}" -u
{shlex.quote(target)} --batch --dump-all'
res_dump = subprocess.run(shlex.split(cmd_dump),
capture_output=True, text=True, timeout=1800)

sorties.append("Résultat du dump de la base de données :\n" +
res_dump.stdout + res_dump.stderr)
else:
    sorties.append("Aucune vulnérabilité détectée dans la base. Le
dump est annulé.")

output = "\n\n".join(sorties)

return render_template("scan.html", target=target, output=output)

```



```
return render_template("scan.html", target=None, output=None)
```

Cette route présente un formulaire pour entrer une URL. Si POST :

- Exécute SQLMap sur cette URL pour tester les injections SQL.
- Si vulnérable, lance un dump de la base.
- Affiche les résultats dans une zone texte sur la page scan.html.
- Envoi un email contenant la vulnérabilité détectée et son type.

Le Code HTML correspondant **scan.html** :

```
{% extends "base.html" %}
{% block title %}SQLMap Scan{% endblock %}
{% block content %}
<h1 class="mb-4">Analyse SQLMap</h1>
<form method="post">
  <div class="mb-3">
    <label class="form-label">URL à tester (GET) :</label>
    <input type="url" name="url" required class="form-control"
      placeholder="https://exemple.com/vuln.php?id=1">
  </div>
  <button class="btn btn-warning">Lancer sqlmap</button>
</form>

{% if output %}
<hr>
<h3>Résultat pour {{ target }}</h3>
<pre class="bg-dark text-light p-3">{{ output }}</pre>
{% endif %}
{% endblock %}
```

Cette page html :

- Hérite du fichier base.html
- Définit le titre de la page
- Champ de saisie d'URL à tester avec SQLMap.
- Bouton qui envoie le formulaire (avec l'URL à scanner)
- Affiche le résultat du scan



Cyber-Hospital

Analyse SQLMap

URL à tester (GET) :

https://exemple.com/vuln.php?id=1

Lancer sqlmap

Page scan

Cyber-Hopital

Analyse SQLMap

URL à tester (GET) :

http://127.0.0.1:5000/patient-insecure?id=1

Lancer sqlmap

Résultat pour <http://127.0.0.1:5000/patient-insecure?id=1>

Étape 1 : Test de vulnérabilité SQL sur l'URL cible...

Résultat du test de vulnérabilité :

```

      H
    [.] {1.9.7.6#dev}
[ _ - ] . ['] | . ' | . |
[ _ _ ] [,] | | | _ , | _ |
      | v... | https://sqlmap.org

```

```
[!] legal disclaimer: Usage of sqlmap for attacking targets witho
```

```
[*] starting @ 14:39:03 /2025-07-25/
```

```
[14:39:08] [INFO] resuming back-end DBMS 'mysql'
```

```
[14:39:08] [INFO] testing connection to the target URL
```



On lance un scan de l'url « <http://127.0.0.1:5000/patient-insecure?id=1> » et on remarque les résultats

```
[*] starting @ 14:39:03 /2025-07-25/
```

```
[14:39:08] [INFO] resuming back-end DBMS 'mysql'
```

```
[14:39:08] [INFO] testing connection to the target URL
```

```
sqlmap resumed the following injection point(s) from stored sessi
```

```
---
```

```
Parameter: id (GET)
```

```
Type: boolean-based blind
```

```
Title: AND boolean-based blind - WHERE or HAVING clause
```

```
Payload: id=1 AND 1296=1296
```

```
Type: error-based
```

```
Title: MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY
```

```
Payload: id=1 AND GTID_SUBSET(CONCAT(0x7170767a71,(SELECT (EL
```

```
Type: time-based blind
```

```
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
```

```
Payload: id=1 AND (SELECT 2887 FROM (SELECT(SLEEP(5)))OkAO)
```

```
Type: UNION query
```

```
Title: Generic UNION query (NULL) - 15 columns
```

```
Payload: id=-3572 UNION ALL SELECT NULL,CONCAT(0x7170767a71,0
```

```
---
```

```
[14:39:08] [INFO] the back-end DBMS is MySQL
```

```
back-end DBMS: MySQL >= 5.6
```

```
[14:39:08] [INFO] fetched data logged to text files under 'C:\Use
```

```
[*] ending @ 14:39:08 /2025-07-25/
```

```
Aucune vulnérabilité détectée dans la base. Le dump est annulé.
```

Puisque cette url est vulnérable le système lance automatiquement un scan de toute la base de données le message affiché au-dessus montre que aucune vulnérabilité est observée sur la base.

Ainsi un fichier log s'est généré contenant les types d'injection SQL auxquels l'URL testée était vulnérable.

Type d'injection	Description
Boolean-based blind	Envoie des requêtes avec des conditions logiques pour inférer des réponses
Error-based	Force des erreurs pour extraire des données via les messages d'erreur
Time-based blind	Utilise la fonction SLEEP pour vérifier les comportements temporels
UNION query	Combine plusieurs requêtes pour extraire des données supplémentaires

Chaque technique inclut :

- **Le type de requête** utilisé (ici GET)
- **Le paramètre ciblé** (id)
- **Le nom du test**
- **Le code envoyé (payload)** pour tester la faille

```

UserssaidmAppDataLocalTemp\sw\h\w\k > 127.0.0.1 > ≡ log
1  sqlmap identified the following injection point(s) with a total of 78 HTTP(s) requests:
2  ---
3  Parameter: id (GET)
4      Type: boolean-based blind
5      Title: AND boolean-based blind - WHERE or HAVING clause
6      Payload: id=1 AND 1132=1132
7
8      Type: error-based
9      Title: MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)
10     Payload: id=1 AND GTID_SUBSET(CONCAT(0x717a6b7171,(SELECT (ELT(6880=6880,1))),0x716b717671),6880)
11
12     Type: time-based blind
13     Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
14     Payload: id=1 AND (SELECT 4925 FROM (SELECT(SLEEP(5)))WkOf)
15
16     Type: UNION query
17     Title: Generic UNION query (NULL) - 15 columns
18     Payload: id=-7408 UNION ALL SELECT NULL,NULL,CONCAT(0x717a6b7171,0x72494d554d48485552785250784b62575
19  ---
20  back-end DBMS: MySQL >= 5.6
21
    
```

Fichier log

3.4 TESTS ET SCENARIOS D'ATTAQUES SIMULEES

La validation de l'efficacité du système de détection a été réalisée à travers des tests de simulation d'attaques fréquentes dans le milieu hospitalier. Les scénarios simulés comprenaient des comportements suspects ou malveillants qui peuvent affecter la confidentialité ou l'intégrité des données médicales. Chacun des tests a ensuite été utilisé pour mesurer la réactivité du système et la précision des alertes.

3.4.1 ACCES HORS HORAIRES

Un des scénarios testés consistait à simuler des accès aux bases de données en dehors des horaires habituels de travail, par exemple durant la nuit ou les week-ends. Il est probable que ces accès peuvent indiquer un comportement anormal ou une tentative d'intrusion. Le système a bien détecté ces événements comme suspects et les a enregistrés dans les journaux d'anomalies, ce qui permis une remontée rapide aux responsables de la sécurité informatique (RSI).

id a...	utilisateur	description	gravite	horodatage
1	dr.ahmed	Accès à la table patients à 03:42 du matin — comportement inhabituel pour un médecin	moyenne	2025-07-10T15:01:02.000Z
2	infirmiere.rania	Consultation de données de consultations à 4h15 — accès anormal en dehors des horaires	moyenne	2025-07-10T15:01:02.000Z
5	dr.ahmed	Accès à la table patients à 03:42 du matin — comportement inhabituel pour un médecin	moyenne	2025-07-10T15:18:21.000Z
6	infirmiere.rania	Consultation de données de consultations à 4h15 — accès anormal en dehors des horaires	moyenne	2025-07-10T15:18:21.000Z

Table : anomalies_detectees

```

exports > rapport_incidents_20250715_010451.csv
1  utilisateur,description,gravite,horodatage
2  test.user,Accès à patients à 2025-07-15 00:54:32,moyenne,2025-07-15 01:01:43
3  test.user,Accès à patients à 2025-07-15 00:55:02,moyenne,2025-07-15 01:01:43
4  test.user,Accès à patients à 2025-07-15 00:55:32,moyenne,2025-07-15 01:01:43
5  test.user,Accès à patients à 2025-07-15 00:56:02,moyenne,2025-07-15 01:01:43
6  test.user,Accès à patients à 2025-07-15 00:56:32,moyenne,2025-07-15 01:01:43

```

rapport_incidents_20250715_010451.csv

3.4.2 TENTATIVES D'ACCES PAR DES UTILISATEURS NON AUTORISES

74	dr.ahmed	Test manuel : accès non autorisé au dossier patient à 02h35	elevee	2025-07-13T22:30:42.000Z
----	----------	---	--------	--------------------------

Table : anomalies_detectees

id_anomalie	utilisateur	description	gravite	horodatage
Filter...	Filter...	Filter...	Filter...	Filter...
9	dr.ahmed	Accès à patients à 2024-07-06 03:42:00	moyenne	2025-07-11T00:14:29.000Z
10	infirmiere.rania	Accès à consultations à 2024-07-06 04:15:00	moyenne	2025-07-11T00:14:29.000Z
11	dr.ahmed	Accès à patients à 2024-07-06 03:42:00	moyenne	2025-07-11T00:18:03.000Z
12	infirmiere.rania	Accès à consultations à 2024-07-06 04:15:00	moyenne	2025-07-11T00:18:03.000Z
13	dr.ahmed	Accès à patients à 2024-07-06 03:42:00	moyenne	2025-07-11T00:21:44.000Z
14	infirmiere.rania	Accès à consultations à 2024-07-06 04:15:00	moyenne	2025-07-11T00:21:44.000Z
15	dr.ahmed	Accès à patients à 2024-07-06 03:42:00	moyenne	2025-07-11T17:39:10.000Z
16	infirmiere.rania	Accès à consultations à 2024-07-06 04:15:00	moyenne	2025-07-11T17:39:10.000Z
17	dr.ahmed	Accès à patients à 2024-07-06 03:42:00	moyenne	2025-07-11T17:46:17.000Z
18	infirmiere.rania	Accès à consultations à 2024-07-06 04:15:00	moyenne	2025-07-11T17:46:17.000Z
19	dr.ahmed	Accès à patients à 2024-07-06 03:42:00	moyenne	2025-07-11T18:00:39.000Z
20	infirmiere.rania	Accès à consultations à 2024-07-06 04:15:00	moyenne	2025-07-11T18:00:39.000Z
21	dr.ahmed	Accès à patients à 2024-07-06 03:42:00	moyenne	2025-07-11T18:00:54.000Z
22	infirmiere.rania	Accès à consultations à 2024-07-06 04:15:00	moyenne	2025-07-11T18:00:54.000Z
23	dr.ahmed	Accès à patients à 2024-07-06 03:42:00	moyenne	2025-07-11T18:01:45.000Z

Table : anomalies_detectees

Ce test voulait voir comment des connexions ou des requêtes simulées peuvent être exécutées par des utilisateurs n'ayant pas de droits d'accès aux tables sensibles comme les tables « patients » et « consultations ». Le système a répondu en bloquant les tentatives et en les signalant comme des anomalies critiques. Cela montre que le système est capable de renforcer les politiques d'accès.

3.4.3 REQUETES SQL SUSPECTES SUR DES TABLES MEDICAUX

3	user.test	Tentative de suppression non autorisée sur la table medecins — utilisateur inconnu	elevée
4	secretaire1	Tentative d'UPDATE sur table patients sans permission — rôle non autorisé	elevée
5	dr.ahmed	Accès à la table patients à 03:42 du matin — comportement inhabituel pour un médecin	moyenne
6	infirmiere.rania	Consultation de données de consultations à 4h15 — accès anormal en dehors des horaires	moyenne
7	user.test	Tentative de suppression non autorisée sur la table medecins — utilisateur inconnu	elevée
8	secretaire1	Tentative d'UPDATE sur table patients sans permission — rôle non autorisé	elevée

Table : anomalies_detectees

Ce test vise à marquer des requêtes SQL suspects comme des anomalies critiques de gravité élevée :

- Si un utilisateur inconnu tente de supprimer des données médicales sensibles cela peut indiquer une tentative de sabotage ou une intrusion.
- Un secrétaire n'a normalement pas les droits pour modifier les dossiers patients. Ce comportement représente une violation des droits d'accès



3.4.4 QU'EST-CE QUE L'INJECTION SQL ?

L'injection SQL est une vulnérabilité où un attaquant interfère avec les requêtes SQL qu'une application fait à la base de données dorsale. En ajoutant des caractères ou du code SQL à un vecteur d'entrée, l'attaquant peut manipuler le comportement de la requête. Les attaques par injection SQL sont parmi les plus fréquentes dans les systèmes de bases de données [1].

3.4.5 IMPACT DES ATTAQUES PAR INJECTION SQL

L'impact varie en fonction du contexte de la vulnérabilité. Il peut être catégorisé en utilisant la triade CIA :

- **Confidentialité** : Visualisation d'informations sensibles comme les noms d'utilisateur et les mots de passe.
- **Intégrité** : Modification des données, comme la mise à jour de l'adresse e-mail d'un autre utilisateur pour pirater son compte.
- **Disponibilité** : Affecter l'accès aux comptes ou provoquer un déni de service.

3.4.6 Principaux types d'injections SQL

1. Injection SQL classique (In-band SQLi)

Définition :

L'injection SQL classique, aussi appelée **In-band SQL Injection**, est la forme la plus simple et la plus courante d'attaque par injection SQL. Dans ce cas, l'attaquant utilise la même voie de communication pour envoyer sa requête malveillante au serveur et pour récupérer les résultats de cette requête.

Comment ça marche ?

Imagine que tu envoies une requête à un site web via un formulaire ou une URL, par exemple pour rechercher un utilisateur. L'attaquant va insérer dans ce champ des morceaux de code SQL spécialement conçus pour manipuler la base de données. Si le site ne filtre pas correctement ces entrées, la base de données exécute ces commandes. L'attaquant voit immédiatement le résultat directement dans la page web.



p02009187@gmail.com

À moi ▼

17:23 (il y a 0 minute)



ALERTE SÉCURITÉ SYSTÈME

=====

URL testée: <http://testphp.vulnweb.com/listproducts.php?cat=1>

Base de données: hopital_db

Niveau de risque: MOYEN

Date/Heure: 2025-07-26 17:23:54

VULNÉRABILITÉ DÉTECTÉE: Injection SQL classique (In-band SQLi)



Actions recommandées:

- Vérifier immédiatement la sécurité du site
- Corriger les vulnérabilités SQL
- Mettre à jour les protections

Ce message a été généré automatiquement par le système de détection.

Email d'alerte reçu à propos de l'injection détectée

Résultat pour <http://testphp.vulnweb.com/listproducts.php?cat=1>

Étape 1 : Test de vulnérabilité SQL sur l'URL cible...

Résultat du test de vulnérabilité :

```

      _H_
      ['']
      {1.9.7.6#dev}
      | - | . [ , ] | . ' | . |
      | _ | _ [ ) ] _ | _ | _ , | _ |
      | _ | V . . . | _ | https://sqlmap.org
  
```

[!] legal disclaimer: Usage of sqlmap for attacking targets witho

[*] starting @ 17:15:06 /2025-07-26/

```

[17:15:07] [INFO] testing connection to the target URL
[17:15:08] [WARNING] there is a DBMS error found in the HTTP resp
[17:15:08] [INFO] checking if the target is protected by some kin
[17:15:08] [INFO] testing if the target URL content is stable
[17:15:08] [INFO] target URL content is stable
[17:15:08] [INFO] testing if GET parameter 'cat' is dynamic
[17:15:08] [WARNING] GET parameter 'cat' does not appear to be dy
[17:15:08] [INFO] heuristic (basic) test shows that GET parameter
[17:15:09] [INFO] testing for SQL injection on GET parameter 'cat
it looks like the back-end DBMS is 'MySQL'. Do you want to skip t
[17:15:09] [INFO] testing 'AND boolean-based blind - WHERE or HAV
[17:15:11] [WARNING] reflective value(s) found and filtering out
[17:15:39] [INFO] testing 'OR boolean-based blind - WHERE or HAVI
[17:15:42] [INFO] GET parameter 'cat' appears to be 'OR boolean-b
[17:15:42] [INFO] testing 'Generic inline queries'
  
```




Sous-types principaux :

- **Error-based SQLi** : L'attaquant exploite les messages d'erreur générés par la base de données pour obtenir des informations. Par exemple, si la requête échoue, la base renvoie un message détaillé qui peut révéler la structure de la base.

Type: error-based

Title: MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)

Payload: artist=1 AND GTID_SUBSET(CONCAT(0x7162707671,(SELECT (ELT(4023=4023,1))),0x7176627071),4023)

- **Union-based SQLi** : L'attaquant utilise l'opérateur UNION pour combiner sa requête malveillante avec la requête originale, afin de récupérer des données supplémentaires dans le même flux de réponse.

Type: UNION query

Title: Generic UNION query (NULL) - 3 columns

Payload: artist=-8498 UNION ALL SELECT NULL,NULL,CONCAT(0x7162707671,0x4f5a5a57475a677165626b6f)

En résumé : C'est comme si tu demandais une info et que la base te la donne direct, mais l'attaquant s'est glissé dans ta demande pour obtenir plus qu'il ne devrait.

2. Injection SQL aveugle (Blind SQLi)

Définition :

L'injection SQL aveugle est une variante plus discrète et difficile à détecter. Ici, la base de données ne renvoie pas directement les résultats de la requête injectée ou les messages d'erreur détaillés. L'attaquant doit donc "deviner" les informations en posant une série de questions oui/non à la base, et analyser les réponses (par exemple, si une page charge ou si elle affiche une erreur générique).

Comment ça marche ?

Au lieu de voir les données directement, l'attaquant pose des questions booléennes en SQL, par exemple :

Est-ce que le premier caractère du nom d'utilisateur est 'A' ?

Si la réponse affecte la page (elle s'affiche ou non, ou une erreur survient), l'attaquant sait que la réponse est "oui". Il répète ce processus caractère par caractère ou bit par bit pour reconstruire les données.



ALERTE SÉCURITÉ - SQL Injection Détectée - ↕ 🖨 🔗

Niveau: MOYEN Boîte de réception x



p02009187@gmail.com

À moi ▾

15:27 (il y a 1 heure)



ALERTE SÉCURITÉ SYSTÈME

=====

URL testée: <http://testphp.vulnweb.com/artists.php?artist=1>

Base de données: hopital_db

Niveau de risque: MOYEN

Date/Heure: 2025-07-26 15:27:31

VULNÉRABILITÉ DÉTECTÉE: Injection SQL aveugle (Blind SQLi)

Actions recommandées:

- Vérifier immédiatement la sécurité du site
- Corriger les vulnérabilités SQL
- Mettre à jour les protections

Ce message a été généré automatiquement par le système de détection.

Email d'alerte reçu à propos de l'injection détectée

Résultat pour <http://testphp.vulnweb.com/artists.php?artist=1>

Étape 1 : Test de vulnérabilité SQL sur l'URL cible...

Résultat du test de vulnérabilité :

```

      _H_
     [']
_ - [ ] [ ] [ ] [ ] [ ] {1.9.7.6#dev}
_ - [ ] [ ] [ ] [ ] [ ]
_ - [ ] [ ] [ ] [ ] [ ]
     [V... [ ] https://sqlmap.org
  
```

[!] legal disclaimer: Usage of sqlmap for attacking targets witho

[*] starting @ 15:27:29 /2025-07-26/

[15:27:30] [INFO] resuming back-end DBMS 'mysql'

[15:27:30] [INFO] testing connection to the target URL

sqlmap resumed the following injection point(s) from stored sessi

Parameter: artist (GET)

Type: boolean-based blind

Title: AND boolean-based blind - WHERE or HAVING clause

Payload: artist=1 AND 5083=5083

Type: error-based

Title: MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY

Payload: artist=1 AND GTID_SUBSET(CONCAT(0x7162707671,(SELECT

Type: time-based blind

Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)

Sous-types :

- **Blind SQLi basé sur les booléens (Boolean-based)** : L'attaquant modifie la requête pour que la réponse soit vraie ou fausse, et observe les différences dans la page web (par exemple, contenu différent, temps de réponse, etc.).

```
sqlmap identified the following injection point(s) with a total of 50 HTTP(s) requests:
---
Parameter: artist (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: artist=1 AND 5083=5083
```

- **Blind SQLi basé sur le temps (Time-based)** : L'attaquant injecte des commandes qui forcent la base à attendre un certain temps avant de répondre (ex : WAITFOR DELAY en SQL Server), ce qui permet de déduire la réponse selon le délai observé.

```
Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: artist=1 AND (SELECT 8508 FROM (SELECT(SLEEP(5)))vdMR)
```

En résumé : C'est un peu comme un détective qui interroge quelqu'un par oui/non pour découvrir un secret, sans jamais le voir directement.

3. Injection SQL hors bande (Out-of-band SQLi)

Définition :

L'injection SQL hors bande est une méthode plus sophistiquée et moins fréquente. Elle survient quand l'attaquant ne peut pas obtenir les résultats directement via la même connexion ou par des réponses visibles, mais utilise un autre canal de communication externe pour récupérer les données.

Comment ça marche ?

L'attaquant injecte une requête qui fait que la base de données envoie une requête ou un message vers un serveur qu'il contrôle (par exemple via DNS, HTTP, ou un autre protocole). La base de données agit donc comme un relais involontaire. Cela permet à l'attaquant de récupérer des informations même quand les autres types d'injection sont bloqués.

Sous-types :



- **DNS-based Out-of-band SQLi** : La base de données est forcée de faire une requête DNS vers un domaine contrôlé par l'attaquant, contenant des données sensibles encodées dans la requête.
- **HTTP-based Out-of-band SQLi** : La base de données envoie une requête HTTP à un serveur distant, qui reçoit les données injectées.

En résumé : C'est comme si tu demandais à quelqu'un de t'envoyer une lettre ou un message à l'extérieur via un canal secret, plutôt que de te répondre en face.

4. RESULTATS OBTENUS ET EVALUATION

Le système développé a permis de démontrer l'intérêt concret d'une solution de détection automatisée des anomalies d'accès aux bases de données médicales dans un contexte hospitalier. À travers différentes phases de tests et de simulations d'accès, les résultats ont montré que l'outil est capable de détecter plusieurs types de comportements suspects, tout en assurant une traçabilité complète des opérations sensibles.

1. PERTINENCE DU SYSTEME DANS UN ENVIRONNEMENT HOSPITALIER

L'installation du système dans un hôpital est très pertinente. Premièrement, les données médicales sont particulièrement sensibles. Deuxièmement, il existe des exigences de conformité réglementaire basées sur la confidentialité, l'intégrité et la traçabilité des accès. Le système a permis :

- De détecter automatiquement les accès anormaux tel que la connexion hors heures de travail, un utilisateur inconnu qui effectue un nombre croissant de requêtes ou le nombre d'accès violé.
- Le système génère un rapport quand une anomalie est détectée. En outre, le fichier CSV est envoyé au RSI par e-mail avec un message d'alerte.
- L'autre avantage du système est son intégration transparente. Il a été bien intégré dans tous les autres systèmes d'hôpital sans perturber le travail personnel des utilisateurs normalement autorisés.
- Permet une réaction rapide en cas d'incident grâce à l'automatisation de l'alerte.

En somme, cet outil constitue une valeur ajoutée stratégique pour renforcer la sécurité des données de santé, souvent ciblées par des actes malveillants.

2. LIMITES ET AXES D'AMELIORATION

Malgré les résultats encourageants, le système présente certaines limites qu'il serait pertinent d'améliorer dans une version future :

- **Portée d'analyse limitée aux logs de base de données** : le système ne prend pas encore en compte les logs applicatifs ou réseau, ce qui limite la vision globale des incidents.



- **Analyse statique des comportements** : le système repose sur des règles fixes (heures, seuils, etc.), alors qu'une approche par « machine learning » pourrait permettre une détection plus fine et évolutive.
- **Interface web basique** : bien que fonctionnelle, l'interface pourrait être enrichie par un tableau de bord interactif, avec des filtres, graphiques, et une gestion des utilisateurs avec rôles.
- **Pas de gestion des incidents post-détection** : l'outil se limite à l'identification et à la notification, mais n'inclut pas encore de système de traitement ou de suivi des anomalies détectées.

3. APPORTS DU STAGE

Durant ce stage d'initiation, j'ai pu développer des compétences pratiques et acquérir une meilleure compréhension du lien entre la cybersécurité et la gestion des bases de données médicales. Les principaux apports se résument comme suit :

1. **Compréhension des enjeux de sécurité dans le domaine médical**
 - Sensibilisation à la protection des données sensibles des patients.
 - Identification des risques liés aux accès non autorisés et aux cyberattaques.
 - Importance de la traçabilité et de la conformité aux normes de sécurité (RGPD, sécurité hospitalière).
2. **Compétences techniques en développement**
 - Mise en place d'un système de détection d'anomalies sur les bases de données médicales.
 - Utilisation de Python pour automatiser l'analyse des logs et détecter les accès suspects.
 - Découverte et manipulation de MySQL pour la gestion des données médicales et des journaux d'accès.
 - Création de triggers, vues et index pour améliorer la performance et la sécurité des bases de données.
3. **Expérience dans l'intégration d'outils de surveillance**
 - Mise en place d'un mécanisme de journalisation (logs) pour suivre les actions des utilisateurs.
 - Conception d'un système d'alertes automatiques (par e-mail) en cas d'accès anormal ou d'injection SQL détectée.
 - Familiarisation avec les principes de SIEM (Security Information and Event Management) appliqués dans un contexte hospitalier.
4. **Développement de l'esprit analytique et méthodologique**
 - Apprentissage de la méthodologie de détection d'incidents de sécurité.
 - Capacité à documenter les anomalies et proposer des recommandations correctives.



CONCLUSION GENERALE

Ce stage au sein de l'établissement hospitalier m'a permis de mettre en pratique mes compétences en développement numérique tout en me familiarisant avec les enjeux cruciaux de la cybersécurité dans un environnement sensible tel qu'un hôpital. Le projet intitulé « Système de détection et de journalisation des accès anormaux aux bases de données médicales » a constitué une expérience enrichissante, tant sur le plan technique que méthodologique.

À travers la conception et la mise en place de mécanismes de surveillance (triggers, logs, vues, scripts d'analyse et alertes automatiques), j'ai pu apporter une solution concrète de détection précoce des comportements suspects ou non conformes. Ce système contribue à renforcer la confidentialité, l'intégrité et la traçabilité des données médicales, tout en respectant les règles d'accès selon les rôles définis.

Cette expérience m'a permis de comprendre l'importance d'une sécurité proactive, en anticipant les risques liés aux accès non autorisés, aux erreurs humaines ou aux tentatives malveillantes. Elle m'a également permis de développer une approche rigoureuse de la documentation, des tests, ainsi qu'un esprit critique face aux anomalies détectées.

En somme, ce projet a été une réelle opportunité de combiner mes compétences en développement et en cybersécurité pour répondre à un besoin réel et concret du milieu hospitalier. Il ouvre également la voie à de futures améliorations, notamment en intégrant de l'intelligence artificielle ou du « machine learning » pour une détection encore plus fine des comportements anormaux.



WEBOGRAPHIE

- [1] https://owasp.org/www-community/attacks/SQL_Injection
- [2] <https://sqlmap.org/>
- [3] <https://www.imsnetworks.com/ressources-et-actual/siem/>
- [4] <https://www.chu-fes.ma/>
- [5] <https://flask.palletsprojects.com/>
- [6] <https://dev.mysql.com/doc/refman/8.0/en/>
- [7] <https://docs.python.org/3/library/smtplib.html>
- [8] <https://openclassrooms.com/fr/courses/1750566-optimisez-la-securite-informatique-grace-au-monitoring>
- [9] <https://notsosecure.com/continuous-security-monitoring>